

Texture Explorer

Texture, bump, displacement and roughness maps

Theory and implementation of an interactive teaching tool

October 7, 2026

Contents

1	Introduction	1
I	Theory	3
2	Texture space and parametric surfaces	4
2.1	Images, texels and normalized coordinates	4
2.1.1	Which way is up?	4
2.2	Parametric surfaces	5
2.2.1	Orientation and mirrored textures	5
2.2.2	Distortion	5
2.3	The four solids	6
2.3.1	Sphere	6
2.3.2	Cylinder	6
2.3.3	Capsule	6
2.3.4	Pyramid	7
2.3.5	Seams	7
3	Sampling textures	8
3.1	Addressing	8
3.2	Nearest and bilinear filtering	8
3.3	Minification and mipmaps	9
3.4	Colour spaces	9
4	Height fields: displacement, bump and normal maps	11
4.1	Displacement mapping	11
4.2	Bump mapping	12
4.2.1	Blinn's derivation	12
4.2.2	In an orthonormal frame	12
4.2.3	The formula used in Texture Explorer	13
4.3	Estimating the slope	13
4.4	Normal maps	13
4.5	Summary	14

5	Tangent space	15
5.1	The tangent frame	15
5.2	Building an orthonormal frame	16
5.3	A worked example	16
5.4	Transforming normals	16
6	Lighting and specular roughness	18
6.1	Reflection from a point light	18
6.2	Diffuse reflection	18
6.3	Specular reflection: microfacets	18
6.3.1	The GGX distribution	19
6.3.2	Masking and shadowing	19
6.3.3	Fresnel reflectance	19
6.4	The complete model	20
6.4.1	The horizon factor	20
II	Implementation	21
7	Architecture of the program	22
7.1	Modules	22
7.2	One frame	22
7.3	A design principle: compute it twice	23
8	Geometry: patches, meshes and tangent frames	24
8.1	Surface points, vertices and patches	24
8.2	The four solids	25
8.3	The tangent frame	27
8.4	Building the mesh	27
8.5	Inverting the texture mapping	28
9	Materials and textures	29
9.1	Collecting the materials	29
9.2	Two copies of every map	29
9.3	Sampling on the CPU	30
9.4	Loading an image into a typeless texture	30
9.5	Loading a material	31
10	The probe	33
10.1	The normal of the bump map	33
10.2	From the texel to the world	33
10.3	What the Inspector shows	34

11 The shaders	36
11.1 The constant buffer	36
11.2 Vertex shader: displacement	37
11.3 The bump normal on the GPU	37
11.4 Pixel shader: bump mapping and lighting	37
12 The renderer and the user interface	40
12.1 Shaders that can be reloaded	40
12.2 The off-screen target	41
12.3 Drawing the scene	42
12.4 The frame of the application	43
12.5 Images that answer the mouse	44
12.6 Filling the constant buffer	45
12.7 The main loop	45
13 Experiments and exercises	47
13.1 Experiments with the program	47
13.2 Programming exercises	47
A The materials	49
B Building this document	50
References	51

Chapter 1

Introduction

A modern real-time material is not one image but several. A colour texture says what the surface looks like under white light; a *bump* or *normal* map adds the small dents and grooves that catch the light; a *displacement* map really moves the surface; and a *roughness* map decides, point by point, whether the surface is glossy or dull. Each of these maps is easy to describe in a sentence and surprisingly subtle to implement, because it touches several ideas of computer graphics at once: the parametrization of surfaces, the sampling of images, differential geometry, coordinate frames, and the physics of reflection.

Texture Explorer is a small interactive program written to make those ideas visible. It shows a material and all its maps, applies them to a simple solid, and lets the student move the camera and the light. Its central feature is a *probe*: pointing with the mouse at any texel of any map, the student sees

- the texture coordinates (u, v) of the texel, normalized to $[0, 1]$;
- the value of every map there: colour, bump height, displacement and roughness;
- the slope of the bump map, and the normal vector it implies, expressed both in the *tangent space* of the surface and in *world* coordinates;
- in the 3D view, the point of the solid that carries the texel, with arrows for the geometric normal $\mathbf{N}$, the bumped normal $\mathbf{n}'$, and the tangent frame $\mathbf{T}$, $\mathbf{B}$.

How this guide is organized

Part I develops the theory that the program illustrates. Chapter 2 introduces texture space and the parametrization of surfaces, using the four solids of the program as examples. Chapter 3 explains how a texture is sampled: texel centres, bilinear filtering, mipmaps and colour spaces. Chapter 4 is the core: height fields used as bump maps and as displacement maps, the derivation of the perturbed normal, and normal maps. Chapter 5 studies the tangent frame and the transformation of normals, and Chapter 6 the microfacet model of reflection through which roughness acts.

Part II walks through the implementation. Every listing in it is extracted automatically from the source files when this document is built, so the code shown is always the code that runs; each listing is labelled with its file and line numbers. Chapter 13 proposes experiments and exercises for a course. For a broader treatment of every topic, see the books by Akenine-Möller et al. [1] and by Pharr et al. [8].

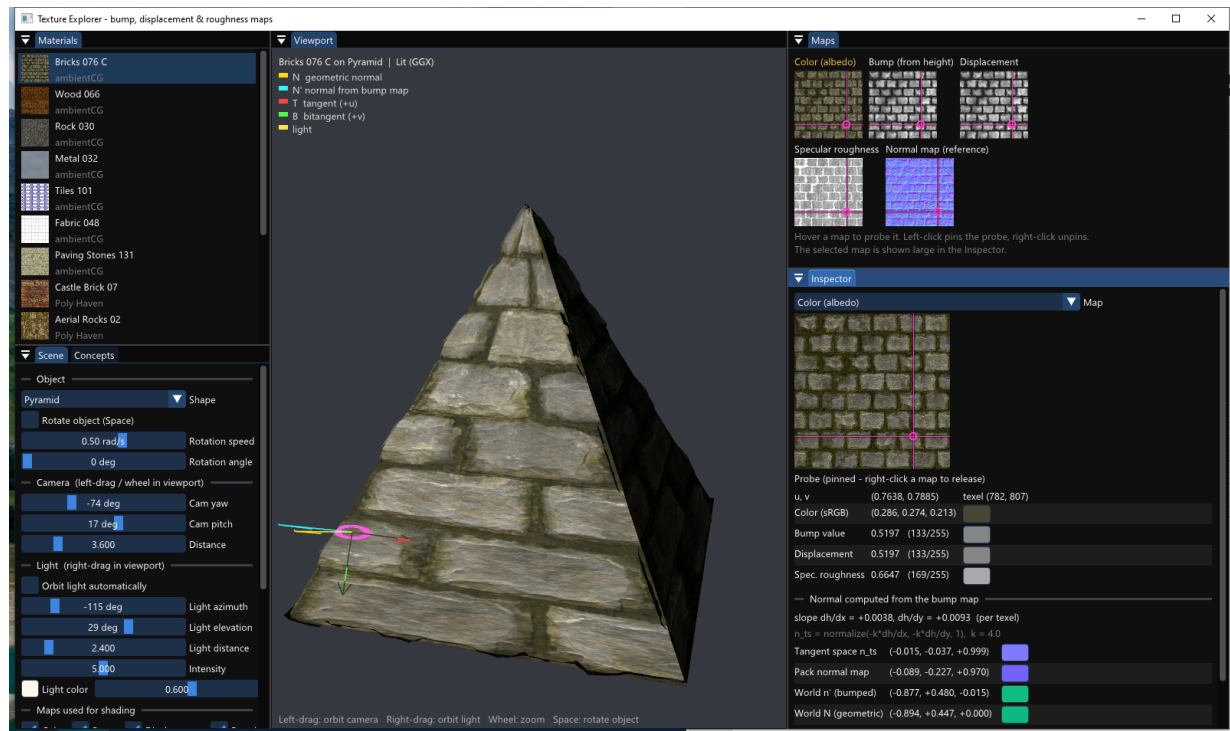


Figure 1.1: Texture Explorer showing a brick material on the pyramid. The probe is pinned on a texel (crosshairs on every map); the Inspector lists its values and normals, and the 3D view marks the same texel with a magenta ring and the arrows **N** (yellow), **n'** (cyan), **T** (red) and **B** (green).

Running the program

The program needs Windows 10 or later and a GPU with Direct3D 11. From the root of the repository:

```
powershell -ExecutionPolicy Bypass -File tools\fetch_textures.ps1 # once
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release
build\Release\TextureExplorer.exe
```

The window is divided into dockable panels: *Materials* (the list of materials and the origin of the selected one), *Scene* (controls), *Concepts* (short explanations), *Viewport* (the 3D view), *Maps* (all the maps of the material) and *Inspector* (the probe). In the viewport, dragging with the left button orbits the camera, dragging with the right button orbits the light, the wheel zooms, and the space bar starts or stops the rotation of the solid. On a map, hovering moves the probe, a left click pins it, and a right click releases it.

Conventions. Vectors are bold: **p**, **N**. Unit vectors are not marked specially. The world is right-handed with y up. Texture space follows Direct3D: (0,0) is the top-left corner of an image and v grows downwards (Section 2.1.1).

Part I

Theory

Chapter 2

Texture space and parametric surfaces

2.1 Images, texels and normalized coordinates

A texture is an image of $W \times H$ *texels*. Shaders do not address texels by integer indices, however, but by *normalized texture coordinates* $(u, v) \in [0, 1]^2$, so that the same coordinates work whatever the resolution of the image. Texel (i, j) , with $0 \leq i < W$ and $0 \leq j < H$, covers the little rectangle

$$\left[\frac{i}{W}, \frac{i+1}{W} \right] \times \left[\frac{j}{H}, \frac{j+1}{H} \right],$$

and its *centre* is at $((i + \frac{1}{2})/W, (j + \frac{1}{2})/H)$. The distinction between the corner and the centre of a texel matters as soon as we interpolate between texels (Chapter 3); the program's probe reports both the normalized coordinates and the integer texel $(\lfloor uW \rfloor, \lfloor vH \rfloor)$.

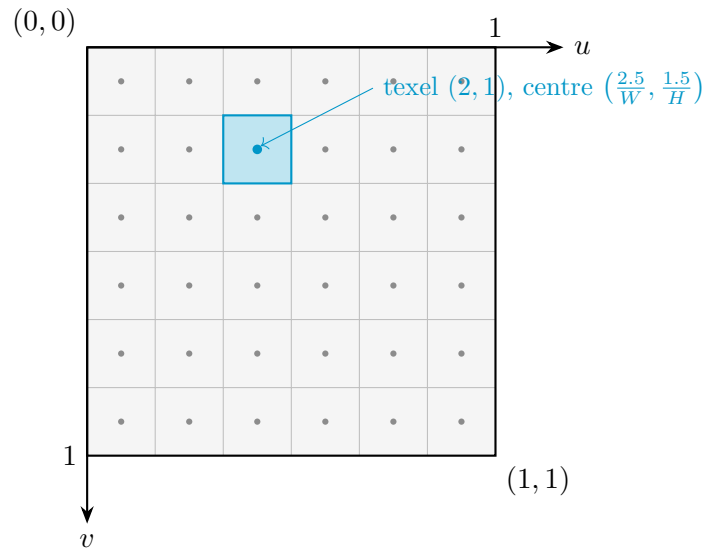


Figure 2.1: Normalized texture space in the Direct3D convention, for an image of $W = H = 6$ texels. The dots are texel centres.

2.1.1 Which way is up?

Two conventions coexist. Direct3D, Vulkan and Metal put $(0, 0)$ at the *top-left* corner of the image, with v growing downwards, which is the order in which image files store their rows. OpenGL puts

$(0, 0)$ at the *bottom-left*, with v growing upwards, like a mathematical graph. The two are related by $v_{\text{GL}} = 1 - v_{\text{D3D}}$. Texture Explorer uses the Direct3D convention throughout (Figure 2.1), which has a pleasant side effect: the mouse position on an image on the screen, where y also grows downwards, converts to (u, v) without any flip.

The convention leaves a trace in every normal map: the green channel stores the component of the normal along $+v$, and $+v$ points down in Direct3D but up in OpenGL. That is why material libraries ship two variants, “NormalDX” and “NormalGL”, whose green channels are inverted with respect to each other (Section 4.4).

2.2 Parametric surfaces

Texture mapping assigns texture coordinates to every point of a surface. For the simple solids of Texture Explorer the natural way to do this is to describe the surface itself as a *parametric surface*, a function from a domain of the plane to space,

$$\mathbf{P} : [0, 1]^2 \rightarrow \mathbb{R}^3, \quad (u, v) \mapsto \mathbf{P}(u, v),$$

and to use the parameters as texture coordinates. (In the program the parameters of a patch are called (a, b) and are mapped affinely to (u, v) ; for this chapter we identify them.)

The two partial derivatives

$$\mathbf{P}_u = \frac{\partial \mathbf{P}}{\partial u}, \quad \mathbf{P}_v = \frac{\partial \mathbf{P}}{\partial v}$$

are tangent to the surface: $\mathbf{P}_u$ points in the direction in which the texture’s u increases, and $\mathbf{P}_v$ in the direction of increasing v . Where they are linearly independent, their cross product is perpendicular to the surface, and the unit normal is

$$\mathbf{N} = \pm \frac{\mathbf{P}_u \times \mathbf{P}_v}{\|\mathbf{P}_u \times \mathbf{P}_v\|}.$$

The sign is chosen so that $\mathbf{N}$ points out of the solid.

2.2.1 Orientation and mirrored textures

Which sign is the outward one? Imagine looking at the surface from outside. For the texture to appear the right way round—not mirrored— u must grow to the viewer’s right and, with the Direct3D convention, v must grow *downwards*. In a right-handed system, right $\times$ down points *into* the surface. Hence, for a correctly oriented texture,

$$\mathbf{P}_u \times \mathbf{P}_v \text{ points inwards, i.e. } \mathbf{T} \times \mathbf{B} = -\mathbf{N} \quad (2.1)$$

where $\mathbf{T}$ and $\mathbf{B}$ are $\mathbf{P}_u$ and $\mathbf{P}_v$ normalized (and made perpendicular, Chapter 5). Every shape of the program was checked against (2.1); if it fails, the texture appears mirrored, text in it would read backwards, and—more subtly—the normals computed from a bump map lean the wrong way.

2.2.2 Distortion

A texture is a flat image, and most surfaces cannot be flattened without stretching. How much a parametrization stretches the texture is measured by the lengths of the derivatives and the angle between them, collected in the *first fundamental form*

$$E = \mathbf{P}_u \cdot \mathbf{P}_u, \quad F = \mathbf{P}_u \cdot \mathbf{P}_v, \quad G = \mathbf{P}_v \cdot \mathbf{P}_v.$$

A small square $du \times dv$ of texture becomes, on the surface, a parallelogram with sides $\sqrt{E} du$ and $\sqrt{G} dv$ and area $\sqrt{EG - F^2} du dv$. The texture keeps its proportions where $E = G$ and $F = 0$ (the map is *conformal* there), and keeps its scale where, in addition, E is constant.

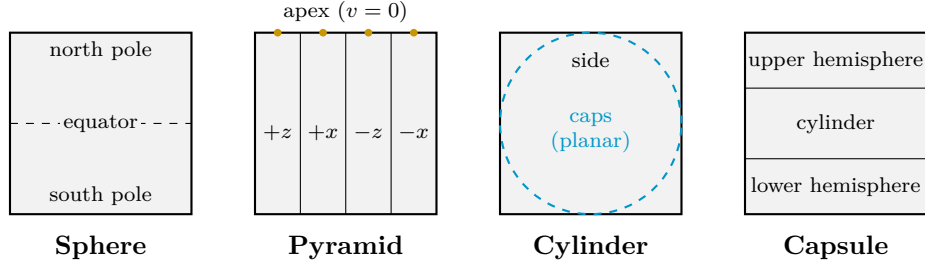


Figure 2.2: How the unit square of texture space is laid out on each solid. The pyramid gives each side face a vertical strip whose top edge collapses to the apex. The cylinder’s caps reuse the coordinates of the side through a planar projection, so they are not part of the invertible chart. On the capsule the bands are proportional to arc length.

2.3 The four solids

Figure 2.2 summarizes how each solid of the program uses texture space. They were chosen because each one illustrates a different aspect of parametrization.

2.3.1 Sphere

With polar angle $\theta = \pi v$ (measured from the north pole) and azimuth $\varphi = 2\pi u$,

$$\mathbf{P}(u, v) = (\sin \theta \cos \varphi, \cos \theta, -\sin \theta \sin \varphi).$$

The minus sign in the third component makes the parametrization satisfy (2.1). The derivatives are

$$\mathbf{P}_u = 2\pi \sin \theta (-\sin \varphi, 0, -\cos \varphi), \quad \mathbf{P}_v = \pi (\cos \theta \cos \varphi, -\sin \theta, -\cos \theta \sin \varphi),$$

so $E = 4\pi^2 \sin^2 \theta$, $F = 0$, $G = \pi^2$. The ratio $\sqrt{E}/\sqrt{G} = 2 \sin \theta$ tells the story of every textured globe: at the equator a square texel becomes a rectangle twice as wide as it is tall, and towards the poles the texels are squeezed horizontally until, at the poles themselves, a whole row of the texture collapses to a point. The program drops the scalar factor $\sin \theta$ from $\mathbf{P}_u$ before using it as a tangent direction, so that the direction stays defined at the poles.

2.3.2 Cylinder

The side of a cylinder of radius r and height H , $\mathbf{P}(u, v) = (r \cos \varphi, H/2 - vH, -r \sin \varphi)$, has $E = (2\pi r)^2$, $F = 0$, $G = H^2$: constant stretch, and conformal when $2\pi r = H$. A cylinder is a *developable* surface: it can be unrolled onto the plane without distortion, which is why labels on cans look perfect.

The caps are discs. Texture Explorer textures them with a *planar projection*: the point $(x, \pm H/2, z)$ of a cap gets the texture coordinates of its position in the square circumscribed about the disc. The cap therefore shows a round piece of the texture, but the same (u, v) now appears on the side *and* on both caps: the texture mapping is no longer one-to-one. The probe of the program resolves this by working only with the side, the *main chart*; the shader’s magenta ring around the probed texel, drawn in texture space, appears on all three patches and makes the ambiguity visible.

2.3.3 Capsule

A capsule—a cylinder of radius r and half-length L capped by two hemispheres—is a surface of revolution, described by its *profile* curve in the half-plane (ρ, y) , ρ being the distance to the

axis. The profile has three pieces: a quarter circle of length $q = \pi r/2$, a segment of length $2L$ and another quarter circle. Texture Explorer makes v proportional to the *arc length* s along the profile,

$$s = v(2q + 2L).$$

With this choice G is constant over the whole capsule—equal steps in v are equal distances on the surface—and the texture flows smoothly from the hemispheres onto the cylinder without being squeezed at the junctions. With $r = L = \frac{1}{2}$ the hemispheres take the bands $v < 0.3055$ and $v > 0.6945$ of the texture (Figure 2.2).

2.3.4 Pyramid

The pyramid shows the problem of texturing a polyhedron. One could repeat the whole texture on each triangular face, but then every texel would appear four times on the solid and the question “where is this texel?” would have four answers. Texture Explorer instead gives each face a vertical strip of width $\frac{1}{4}$ in texture space. Face i goes from the apex $\mathbf{A}$ to the base edge $c_i c_{i+1}$:

$$\mathbf{P}(s, t) = \mathbf{A} + t(\text{lerp}(c_i, c_{i+1}, s) - \mathbf{A}), \quad u = \frac{i+s}{4}, \quad v = t.$$

The price is visible in the program. At the base, $\|\mathbf{P}_u\| = 4 \cdot 1.6 = 6.4$ while $\|\mathbf{P}_v\| \approx 1.79$, so a square texel becomes a rectangle about 3.6 times wider than tall; towards the apex the rows shrink until the strip’s top edge collapses to a point. This is a good occasion to discuss why real models are *unwrapped* into UV layouts by artists, and why those layouts are made of many separate pieces (*charts* or *islands*) with seams between them.

2.3.5 Seams

Every parametrization of a closed surface has seams. On the sphere, the cylinder and the capsule, the line $u = 0$ and the line $u = 1$ are the same curve on the solid. A mesh built on a grid of parameters therefore has two rows of vertices there, at the same positions but with different texture coordinates, $u = 0$ and $u = 1$. The texture must *tile* (its left and right borders must match) for the seam to be invisible; the materials of the program are designed to tile.

Try it in the program. Choose the view “UV coordinates” in the Scene panel. Red encodes u and green encodes v . On the sphere, follow the red gradient around the equator and find the seam; on the pyramid, notice that each face has its own red gradient from $0.25k$ to $0.25(k+1)$; on the cylinder, compare the side and the caps.

Chapter 3

Sampling textures

A shader asks for the value of a texture at an arbitrary point (u, v) , which almost never falls on a texel centre. How the answer is computed—*texture filtering*—affects both the quality of the image and, in Texture Explorer, the agreement between the numbers the probe computes on the CPU and the pixels the GPU draws.

3.1 Addressing

Coordinates outside $[0, 1]$ are folded back according to an *addressing mode*. Texture Explorer uses *wrap* (repeat): the texture tiles the plane, and texel indices are taken modulo the size of the image. In C++, where the remainder of a negative number is negative, the correct index is

$$\text{wrap}(i, n) = ((i \bmod n) + n) \bmod n.$$

Wrapping matters even inside $[0, 1]$: the texels on the left border must be interpolated with those on the right border, which is what makes tiling textures seamless.

3.2 Nearest and bilinear filtering

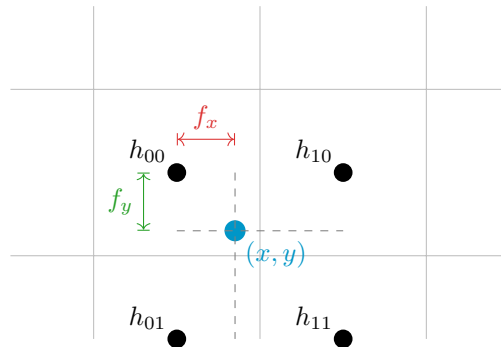


Figure 3.1: Bilinear filtering interpolates the four texel centres that surround the sample point, with weights given by the fractional offsets f_x, f_y .

Nearest filtering returns the texel that contains the point; magnified, the texture looks blocky. *Bilinear* filtering interpolates between the four nearest texel centres. In texel units the point is at $x = uW - \frac{1}{2}$, $y = vH - \frac{1}{2}$ (the $-\frac{1}{2}$ moves the origin to the first texel's centre); with $x_0 = \lfloor x \rfloor$, $y_0 = \lfloor y \rfloor$ and the fractions $f_x = x - x_0$, $f_y = y - y_0$,

$$h(x, y) = (1 - f_y)[(1 - f_x)h_{00} + f_x h_{10}] + f_y[(1 - f_x)h_{01} + f_x h_{11}],$$

where h_{ab} is the texel $(x_0 + a, y_0 + b)$ (Figure 3.1). Texture Explorer implements exactly this formula on the CPU (Section 9.3), so that the probe reads the same values as the GPU. The result is continuous but its derivative is not: it jumps at texel centres, which will matter when we differentiate height maps.

3.3 Minification and mipmaps

When the texture is seen from far away, one pixel of the screen covers many texels. Sampling just one point of that footprint is *aliasing*: fine detail turns into noise that shimmers when the camera moves. The standard remedy, *mipmapping* [11], precomputes a pyramid of images, each half the size of the previous one and obtained by averaging 2×2 blocks, and chooses for each pixel the level whose texels are about the size of the pixel. If ρ is the number of texels of level 0 that the pixel spans, measured with the screen-space derivatives of (uW, vH) ,

$$\rho = \max\left(\left\|\frac{\partial(uW, vH)}{\partial x}\right\|, \left\|\frac{\partial(uW, vH)}{\partial y}\right\|\right), \quad \lambda = \log_2 \rho,$$

the GPU samples level λ , interpolating between the two nearest levels (*trilinear* filtering). When the footprint is long and thin—a surface seen at a grazing angle—a single level is either too blurry along one direction or aliased along the other; *anisotropic* filtering takes several samples along the long axis. Texture Explorer creates every texture with its complete mipmap chain (generated by the GPU) and uses a sampler with $8\times$ anisotropic filtering.

Mipmaps in a vertex shader. The level λ is computed from the difference between neighbouring *pixels*, which only exist in the pixel shader. A vertex shader that reads a texture—as displacement mapping does—must choose the level itself. The right choice depends on how densely the mesh samples the texture: Texture Explorer’s meshes have 256×256 cells per patch and its maps 1024×1024 texels, so each cell spans four texels and the vertex shader reads level $\log_2(1024/256) = 2$. Reading level 0 instead would make the displaced surface jitter as tiny height details fall between vertices.

Derivatives of a texture. The bump-mapping code of Chapter 4 samples the bump map one texel to each side of the point, always at level 0, so that the CPU and the GPU compute the same slope. This is simple and exact but aliases when the solid is small on the screen; a production renderer would scale the offsets with the mip level, or use a normal map with its own mipmaps.

3.4 Colour spaces

The values stored in an 8-bit colour image are not proportional to light. They are encoded with the *sRGB transfer function*, which spends more of the 256 codes on dark tones, where the eye is more sensitive. Decoding a stored value $C \in [0, 1]$ to linear light gives

$$C_{\text{lin}} = \begin{cases} C/12.92, & C \leq 0.04045, \\ ((C + 0.055)/1.055)^{2.4}, & C > 0.04045, \end{cases}$$

which is close to the power law $C^{2.2}$ (Figure 3.2).

Lighting computations add and multiply amounts of light, so they must work with linear values. Hence the rules that Texture Explorer follows:

- The *colour* map is stored in sRGB and must be decoded before lighting. The GPU does it for free when the texture is read through a view with an `_SRGB` format.

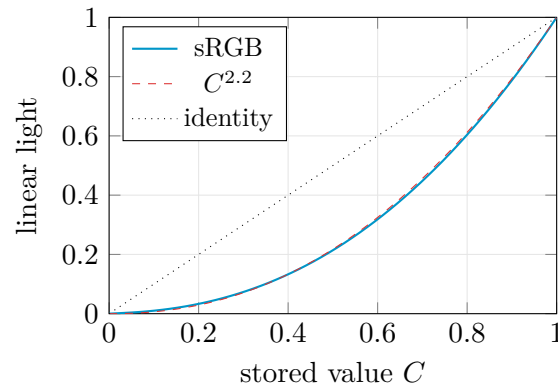


Figure 3.2: The sRGB decoding curve and its usual approximation.

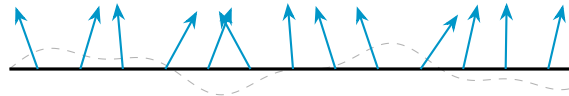
- *Data* maps—bump, displacement, roughness, normal—store numbers, not colours, and must *not* be decoded. They are read through a plain `UNORM` view, which returns the stored byte divided by 255.
- The scene is rendered into an sRGB target: the GPU encodes the linear result when it writes each pixel, and also when it averages the samples of multisample antialiasing, which is therefore done correctly in linear light.
- The user interface, which works with display values, reads both the textures and the rendered scene through plain views, so that the stored values reach the screen untouched.

One texture, two interpretations: Direct3D allows this when the texture is created with a *typeless* format (`R8G8B8A8_TYPELESS`) and each *view* specifies how to read it (Section 9.4).

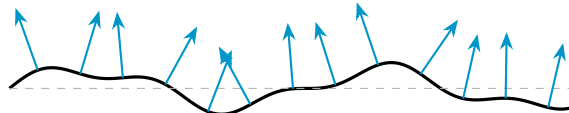
Chapter 4

Height fields: displacement, bump and normal maps

A grey image $h(u, v) \in [0, 1]$ can describe the relief of a surface: light texels are high, dark texels are low. Such an image is a *height field*, and there are two very different ways to use it. A *displacement map* moves the surface; a *bump map* leaves the surface where it is and only changes the normal used for lighting, so that the surface *looks* as if it had been moved. The materials of Texture Explorer come with a height map labelled “displacement”, and the program uses the same image in both roles; switching each role on and off separately is the quickest way to understand the difference (Figure 4.1).



bump mapping: the surface stays flat; only the normals tilt, following the slope of h (dashed)



displacement mapping: the surface itself moves; silhouette and occlusion change

Figure 4.1: The same height field (here a sum of two sines) used in its two roles, seen in cross-section. With bump mapping the normals are exactly those of the displaced surface, but they are attached to the undisplaced one.

4.1 Displacement mapping

Let $\mathbf{P}(u, v)$ be a surface with unit normal $\mathbf{N}(u, v)$. Displacing it by the height field gives the new surface

$$\mathbf{P}'(u, v) = \mathbf{P}(u, v) + s(h(u, v) - m)\mathbf{N}(u, v), \quad (4.1)$$

where s is the *displacement scale*, a length, and m the *mid-level*, the height that leaves the surface in place. With the program’s default $m = \frac{1}{2}$, dark texels push the surface in and light texels push it out; with $m = 0$ the surface only grows.

Displacement mapping was introduced by Cook [3]. On the GPU it is applied per vertex in the vertex shader (Section 11.2). This is the most direct technique and has two consequences worth remembering.

- *The mesh must be dense.* A vertex samples the height field once; detail between vertices is lost. Texture Explorer builds every patch as a grid of 256×256 cells, about 66 000 vertices, and samples the height at the matching mipmap level (Chapter 3). Production renderers use hardware tessellation to make the mesh dense only where it is needed.
- *Normals are not updated.* Moving the vertices does not change the normals that the mesh carries. A displaced surface shaded with its old normals has the right silhouette and the wrong lighting. The fix is to compute the normal of the displaced surface, which is exactly what bump mapping does—which is why displacement and bump (or normal) maps are almost always used together.

Cracks. Equation (4.1) pushes each point along *its* normal. Where a solid has a crease—the edges of the pyramid, the rims of the cylinder—the two patches that meet there have different normals, and vertices at the same position move in different directions: the surface tears open. Texture Explorer leaves the cracks visible on purpose. The usual remedies are to displace along an averaged normal at the crease, or to model creases with a small bevel.

4.2 Bump mapping

In 1978 James Blinn observed that the appearance of small wrinkles is due almost entirely to the way they tilt the normal, not to the way they move the surface [2]. So we can *compute* the normal of the displaced surface (4.1) and use it to shade the *undisplaced* one.

4.2.1 Blinn's derivation

Differentiating (4.1) (and writing $\tilde{h} = s(h - m)$),

$$\mathbf{P}'_u = \mathbf{P}_u + \tilde{h}_u \mathbf{N} + \tilde{h} \mathbf{N}_u, \quad \mathbf{P}'_v = \mathbf{P}_v + \tilde{h}_v \mathbf{N} + \tilde{h} \mathbf{N}_v.$$

For small bumps $\tilde{h}$ itself is tiny while its derivatives $\tilde{h}_u, \tilde{h}_v$ may not be (a small but steep groove), so the terms $\tilde{h} \mathbf{N}_u$ and $\tilde{h} \mathbf{N}_v$ are dropped. The normal of the displaced surface is then parallel to

$$\mathbf{P}'_u \times \mathbf{P}'_v \approx \mathbf{P}_u \times \mathbf{P}_v + \tilde{h}_u (\mathbf{N} \times \mathbf{P}_v) + \tilde{h}_v (\mathbf{P}_u \times \mathbf{N}),$$

(the term $\tilde{h}_u \tilde{h}_v \mathbf{N} \times \mathbf{N}$ vanishes), which is Blinn's perturbed normal.

4.2.2 In an orthonormal frame

Suppose that, at the point, $\mathbf{P}_u = a \mathbf{T}$ and $\mathbf{P}_v = b \mathbf{B}$ with $\mathbf{T}$ and $\mathbf{B}$ orthonormal and perpendicular to $\mathbf{N}$. Then $\tilde{h}_u/a$ is the rate of change of the height per unit *length* along $\mathbf{T}$, and similarly $\tilde{h}_v/b$ along $\mathbf{B}$. Substituting and simplifying—the result is the same whether the frame is right- or left-handed—gives

$$\mathbf{n}' \propto \mathbf{N} - \frac{\tilde{h}_u}{a} \mathbf{T} - \frac{\tilde{h}_v}{b} \mathbf{B}. \quad (4.2)$$

A geometric argument reaches the same result without cross products, and without worrying about handedness. In the frame $(\mathbf{T}, \mathbf{B}, \mathbf{N})$ the displaced surface is locally the graph $z = g(x, y)$, whose tangent vectors are $(1, 0, g_x)$ and $(0, 1, g_y)$. The vector $(-g_x, -g_y, 1)$ is perpendicular to both:

$$(-g_x, -g_y, 1) \cdot (1, 0, g_x) = -g_x + g_x = 0, \quad (-g_x, -g_y, 1) \cdot (0, 1, g_y) = -g_y + g_y = 0.$$

So the normal leans *against* the slope: where the height increases to the right, the normal tilts to the left. On a flat region $g_x = g_y = 0$ and the normal is $(0, 0, 1)$, the unperturbed $\mathbf{N}$.

4.2.3 The formula used in Texture Explorer

Texture Explorer measures the slope in *texels*: h_x is the change of the bump map per texel in the u direction and h_y per texel in the v direction. A single *bump strength* k , set with a slider, converts it into a slope of the surface. The tangent-space normal is

$$\mathbf{n}_{\text{ts}} = \frac{(-k h_x, -k h_y, 1)}{\|(-k h_x, -k h_y, 1)\|}. \quad (4.3)$$

Comparing with (4.2), $k h_x$ plays the role of $\tilde{h}_u/a$, so $k = sW/a$ where W is the width of the texture in texels: k hides the displacement scale *and* the stretch $a = \|\mathbf{P}_u\|$ of the parametrization. Using one k for the whole solid is a deliberate simplification: the bumps look equally deep everywhere, even where the texture is stretched, as on the pyramid near its base. A physically exact bump map would divide by the local stretch.

4.3 Estimating the slope

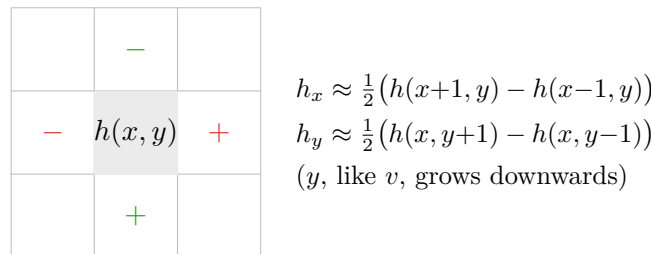


Figure 4.2: The central-difference stencil. The texel itself does not take part.

The derivatives of the bump map are estimated by *central differences*, one texel to each side (Figure 4.2). Taylor's theorem shows why they are preferable to the simpler forward difference $h(x+1) - h(x)$:

$$\frac{h(x+1) - h(x-1)}{2} = h'(x) + \frac{h'''(x)}{6} + \dots, \quad h(x+1) - h(x) = h'(x) + \frac{h''(x)}{2} + \dots$$

The central difference is exact for quadratics and does not shift the normal by half a texel. Both the shader and the CPU probe take the four samples with bilinear filtering at offsets of exactly one texel, so they agree to rounding error.

Try it in the program. Pin the probe on the edge of a brick. Read the slopes h_x , h_y and the tangent-space normal in the Inspector and check equation (4.3) by hand: with $k = 4$, $h_x = 0.0038$ and $h_y = 0.0093$ give $\mathbf{n}_{\text{ts}} \approx (-0.015, -0.037, 0.999)$. Then change the bump strength and watch the cyan arrow $\mathbf{n}'$ in the 3D view lean further.

4.4 Normal maps

Computing derivatives in the shader costs four texture reads per pixel. A *normal map* stores the result instead: each texel holds $\mathbf{n}_{\text{ts}}$, encoded as a colour by

$$rgb = \frac{1}{2} \mathbf{n}_{\text{ts}} + \frac{1}{2}, \quad \mathbf{n}_{\text{ts}} = 2 \, rgb - 1.$$

Because the normals of a mostly flat surface are close to $(0, 0, 1)$, normal maps have the characteristic lavender colour $(0.5, 0.5, 1)$. They are usually not computed from a height map but *baked*

from a detailed (“high-poly”) model onto a simple one, and can represent details that are not height fields at all, such as overhangs.

Two conventions exist, differing in the sign of the green channel (Section 2.1.1): *DirectX* normal maps store the component along $+v$ with v downwards, *OpenGL* ones along $+v$ with v upwards. Using a map of the wrong convention makes the relief look lit from below. Texture Explorer’s tangent-space y axis is $+v$ downwards, so it decodes the DirectX variant directly; the Inspector shows it next to the normal computed from the bump map. The numbers differ—the authors of the material baked their map with their own strength and from their own high-resolution geometry—but the direction in which the normal leans agrees, which is a good check of the conventions.

Normal maps have a subtle filtering problem: averaging unit vectors (as mipmapping does) produces shorter vectors, and renormalizing them loses the information that the surface was bumpy at that scale. Advanced techniques (Toksvig’s, or LEAN mapping) turn that lost length into extra roughness.

4.5 Summary

	Bump map	Normal map	Displacement map
Stores	height h	tangent-space normal $\mathbf{n}_{ts}$	height h
Changes the shading normal	yes, from the slope of h	yes, directly	no (unless normals are recomputed)
Changes silhouette and occlusion	no	no	yes
Cost	4 extra samples per pixel	1 sample per pixel	dense mesh or tessellation
Typical artefacts	flat silhouettes; light leaking on the dark side	same, plus wrong green convention	cracks at creases; swimming if under-sampled
In Texture Explorer	BumpNormalTS in the pixel shader	shown for comparison only	VSMain in the vertex shader

Table 4.1: Three ways to use relief in a material.

Bump and normal maps share one more artefact. A perturbed normal may face the light at points where the real surface faces away from it, so the dark side of the solid sparkles. Texture Explorer attenuates the lighting with a steep ramp of the *geometric* cosine $\mathbf{N} \cdot \mathbf{L}$ (Section 6.4.1), a crude form of self-shadowing.

Chapter 5

Tangent space

Equation (4.3) gives the bumped normal in coordinates relative to the surface. To light the surface we need it in the same coordinate system as the light and the camera. This chapter is about the frame that connects the two, and about transforming normals in general.

5.1 The tangent frame

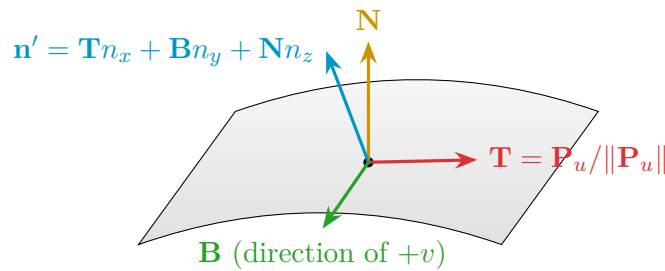


Figure 5.1: The tangent frame at a point of a surface: $\mathbf{T}$ along increasing u , $\mathbf{B}$ along increasing v , $\mathbf{N}$ outwards. A tangent-space vector (n_x, n_y, n_z) is the combination $n_x \mathbf{T} + n_y \mathbf{B} + n_z \mathbf{N}$.

At each point of a textured surface we attach three unit vectors (Figure 5.1): the *tangent* $\mathbf{T}$, in the direction of increasing u ; the *bitangent* $\mathbf{B}$ (sometimes, inaccurately, called the binormal), in the direction of increasing v ; and the normal $\mathbf{N}$. A vector with tangent-space coordinates $\mathbf{n}_{ts} = (n_x, n_y, n_z)$ is, in object coordinates,

$$\mathbf{n}_{obj} = n_x \mathbf{T} + n_y \mathbf{B} + n_z \mathbf{N} = \underbrace{\begin{pmatrix} \mathbf{T} & \mathbf{B} & \mathbf{N} \end{pmatrix}}_{\text{TBN}} \begin{pmatrix} n_x \\ n_y \\ n_z \end{pmatrix}.$$

This is the famous *TBN matrix*. When its columns are orthonormal, its inverse is its transpose, so going back—from object space to tangent space, as some techniques do with the light vector—costs three dot products.

The tangent frame is the reason a single bump or normal map works on any shape: the map describes the relief *relative to the surface*, and the frame carries it onto whatever surface it is applied to.

5.2 Building an orthonormal frame

The derivatives $\mathbf{P}_u$ and $\mathbf{P}_v$ of a parametrization are tangent to the surface but in general neither unit nor perpendicular (think of the pyramid, Chapter 2). Texture Explorer builds an orthonormal frame by the *Gram–Schmidt* process:

$$\begin{aligned}\mathbf{N} &= \text{the unit outward normal,} \\ \mathbf{T} &= \frac{\mathbf{P}_u - (\mathbf{P}_u \cdot \mathbf{N}) \mathbf{N}}{\|\mathbf{P}_u - (\mathbf{P}_u \cdot \mathbf{N}) \mathbf{N}\|}, \\ \mathbf{B} &= \sigma (\mathbf{N} \times \mathbf{T}), \quad \sigma = \text{sign}((\mathbf{N} \times \mathbf{T}) \cdot \mathbf{P}_v).\end{aligned}$$

The sign σ —stored as the fourth component w of the vertex tangent—records the *handedness* of the frame. It is needed because a texture can be applied mirrored; with the conventions of this program, (2.1) gives $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ and so $\sigma = -1$ everywhere. Storing $\mathbf{T}$ and σ instead of $\mathbf{T}$ and $\mathbf{B}$ saves two floats per vertex.

After interpolation. The rasterizer interpolates $\mathbf{N}$ and $\mathbf{T}$ linearly across each triangle. Interpolated unit vectors are shorter than one and no longer perpendicular, so the pixel shader repeats the Gram–Schmidt step before using them. In production, normal maps are baked against a precisely specified tangent frame—the *MikkTSpace* convention [7]—and the renderer must reproduce exactly the same frame, or the baked normals come out slightly wrong.

5.3 A worked example

Take the sphere of Chapter 2 at the centre of the texture, $u = v = \frac{1}{2}$, so $\theta = \pi/2$ and $\varphi = \pi$. Then

$$\begin{aligned}\mathbf{P} &= (-1, 0, 0) = \mathbf{N}, \\ \mathbf{T} &= (-\sin \varphi, 0, -\cos \varphi) = (0, 0, 1), \\ \mathbf{B} &= (\cos \theta \cos \varphi, -\sin \theta, -\cos \theta \sin \varphi) = (0, -1, 0).\end{aligned}$$

Check the orientation: $\mathbf{T} \times \mathbf{B} = (0, 0, 1) \times (0, -1, 0) = (1, 0, 0) = -\mathbf{N}$. A tangent-space normal $\mathbf{n}_{ts} = (0.1, 0, 0.995)$, leaning towards $+u$, becomes

$$\mathbf{n}' = 0.1 \mathbf{T} + 0.995 \mathbf{N} = (-0.995, 0, 0.1)$$

in object space: it leans towards $+z$, the direction in which u grows at that point. This is exactly what the Inspector shows when the probe is placed there (with the solid not rotated).

5.4 Transforming normals

Points and tangent vectors of an object are transformed to the world by its world matrix M (more precisely, by its upper-left 3×3 part for vectors). Normals are different. A normal is defined by being perpendicular to all tangent vectors $\mathbf{t}$: $\mathbf{n} \cdot \mathbf{t} = 0$. If tangents are transformed as $\mathbf{t}' = M\mathbf{t}$, the transformed normal must satisfy $\mathbf{n}' \cdot M\mathbf{t} = 0$ for every $\mathbf{t}$, which holds for

$$\mathbf{n}' = M^{-\text{T}} \mathbf{n}, \quad \text{since} \quad (M^{-\text{T}} \mathbf{n}) \cdot (M\mathbf{t}) = \mathbf{n}^{\text{T}} M^{-1} M \mathbf{t} = \mathbf{n} \cdot \mathbf{t} = 0.$$

For a non-uniform scale, $M^{-\text{T}} \neq M$ and using M for normals is a classic bug: the normals of a squashed sphere would lean the wrong way. For a *rotation*, $M^{-\text{T}} = M$, and for a uniform scale they differ only by a factor that normalization removes. Texture Explorer’s world matrix is a

pure rotation about the vertical axis (the rotation animation), so it transforms points, tangents and normals with the same matrix.

Try it in the program. Pin the probe, then start the rotation of the object (space bar). The world-space vectors in the Inspector change continuously, while the tangent-space normal stays the same: the relief is a property of the surface, the world-space normal a property of the surface *and* its placement in the world.

Chapter 6

Lighting and specular roughness

The bumped normal and the roughness map only become visible through the lighting model. Texture Explorer uses the microfacet model that real-time graphics has adopted as its standard [4, 10, 6]. This chapter states it, explains each factor, and shows where roughness enters.

6.1 Reflection from a point light

The program lights the solid with a single point light at position $\mathbf{p}_L$ with colour and intensity E . At a surface point $\mathbf{x}$ with (bumped) normal $\mathbf{n}$, let

$$\mathbf{L} = \frac{\mathbf{p}_L - \mathbf{x}}{\|\mathbf{p}_L - \mathbf{x}\|}, \quad \mathbf{V} = \frac{\mathbf{p}_{\text{eye}} - \mathbf{x}}{\|\mathbf{p}_{\text{eye}} - \mathbf{x}\|}$$

be the directions to the light and to the eye. The reflected radiance is

$$L_o = f(\mathbf{L}, \mathbf{V}) E (\mathbf{n} \cdot \mathbf{L})^+ + L_{\text{ambient}},$$

where f is the *bidirectional reflectance distribution function* (BRDF) and $(\cdot)^+$ clamps negative values to zero. (A physical point light would also fall off as the inverse square of the distance; the program omits it so that orbiting the light does not change the brightness.) Every $\mathbf{n}$ in this chapter is the normal from the bump map: that is how bump mapping changes the image.

6.2 Diffuse reflection

Light that enters the material, scatters inside and comes out again leaves in all directions equally. That is Lambert's law, with BRDF

$$f_d = \frac{\rho}{\pi},$$

where ρ is the *albedo*, read from the colour map and decoded from sRGB. The $1/\pi$ makes the surface reflect at most all the light it receives.

6.3 Specular reflection: microfacets

A glossy surface is modelled as a multitude of microscopic mirrors, the *microfacets*, whose orientations vary around the normal $\mathbf{n}$ (Figure 6.1). A mirror reflects light from $\mathbf{L}$ into $\mathbf{V}$ only if its normal is the *half vector*

$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{\|\mathbf{L} + \mathbf{V}\|}.$$

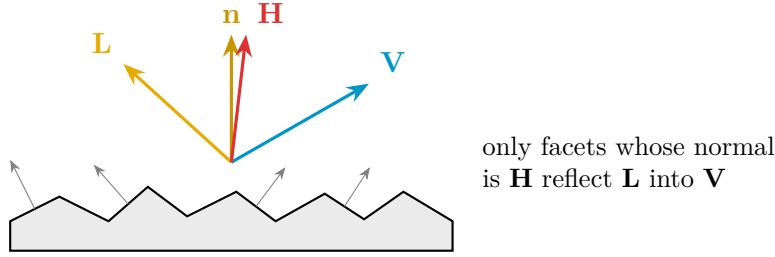


Figure 6.1: The microfacet view of a rough surface: many tiny mirrors (grey normals). The half vector $\mathbf{H}$ is the facet orientation that reflects light from $\mathbf{L}$ towards $\mathbf{V}$.

The Cook–Torrance specular BRDF [4] counts those mirrors:

$$f_s(\mathbf{L}, \mathbf{V}) = \frac{D(\mathbf{H}) G(\mathbf{L}, \mathbf{V}) F(\mathbf{V}, \mathbf{H})}{4 (\mathbf{n} \cdot \mathbf{L}) (\mathbf{n} \cdot \mathbf{V})}.$$

- D , the *normal distribution function*, is the density of microfacets oriented along $\mathbf{H}$. It is where **roughness** enters.
- G , the *geometric* or *masking-shadowing* term, is the fraction of those microfacets that are visible both from the light and from the eye.
- F , the *Fresnel* term, is the fraction of light that a mirror reflects, which grows towards grazing angles.

6.3.1 The GGX distribution

Texture Explorer uses the GGX (Trowbridge–Reitz) distribution [10],

$$D(\mathbf{H}) = \frac{\alpha^2}{\pi ((\mathbf{n} \cdot \mathbf{H})^2 (\alpha^2 - 1) + 1)^2}, \quad \alpha = r^2,$$

where $r \in [0, 1]$ is the value of the roughness map. For small α the distribution is a tall, narrow peak around $\mathbf{n}$: nearly all facets face the same way and the highlight is small and bright. As α grows the peak spreads and lowers; at $\alpha = 1$, $D = 1/\pi$ is constant and there is no highlight at all, only a sheen (Figure 6.2). GGX has longer “tails” than older models such as Blinn–Phong, which gives highlights the soft halo seen on real materials. The remapping $\alpha = r^2$, popularized by Disney and Epic Games [6], makes equal steps of r look like equal steps of glossiness, which is better both for artists painting roughness maps and for a slider.

6.3.2 Masking and shadowing

On a rough surface some facets hide others. The program uses Schlick’s approximation of Smith’s masking function, with the constant $k = (r + 1)^2/8$ recommended for analytic lights [6]; Heitz [5] explains the theory behind these functions:

$$G = G_1(\mathbf{n} \cdot \mathbf{V}) G_1(\mathbf{n} \cdot \mathbf{L}), \quad G_1(c) = \frac{c}{c(1 - k) + k}.$$

6.3.3 Fresnel reflectance

Schlick’s approximation [9] interpolates between the reflectance at normal incidence, F_0 , and total reflection at grazing incidence:

$$F = F_0 + (1 - F_0) (1 - \mathbf{V} \cdot \mathbf{H})^5.$$

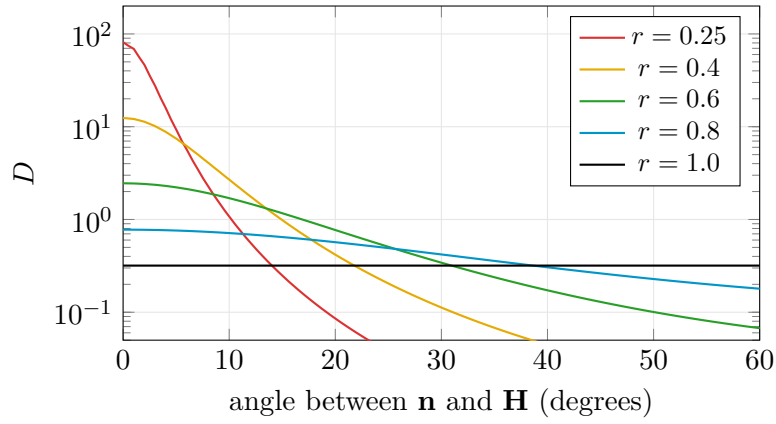


Figure 6.2: The GGX distribution D as a function of the angle between the normal and the half vector, for several values r of the roughness map ($\alpha = r^2$). Note the logarithmic scale: the peak value $1/(\pi\alpha^2)$ is about 81 at $r = 0.25$ but only 2.5 at $r = 0.6$.

For dielectrics (stone, wood, plastic) $F_0 \approx 0.04$ and is colourless; for metals F_0 is high and coloured. The *metallic workflow* interpolates between the two with a *metallic* parameter μ : $F_0 = \text{lerp}(0.04, \rho, \mu)$, and metals have no diffuse reflection.

6.4 The complete model

Putting the pieces together, the pixel shader computes

$$L_o = \left[(1 - F)(1 - \mu) \frac{\rho}{\pi} + \frac{D G F}{4 (\mathbf{n} \cdot \mathbf{L})(\mathbf{n} \cdot \mathbf{V})} \right] E (\mathbf{n} \cdot \mathbf{L})^+ \eta + a \rho,$$

where the factor $1 - F$ removes from the diffuse part the light already reflected specularly, a is a small ambient term, and η is the horizon factor of the next section.

6.4.1 The horizon factor

The bumped normal $\mathbf{n}$ can face the light even where the *geometric* normal $\mathbf{N}$ faces away from it; the result is light on the dark side of the solid, which looks wrong. Texture Explorer multiplies by

$$\eta = \text{saturne}(8 (\mathbf{N} \cdot \mathbf{L})),$$

which fades the lighting out over the last few degrees before the geometric terminator. It is a cheap stand-in for the self-shadowing of the relief.

Try it in the program. Choose a material whose roughness tile shows strong contrast (dark texels are glossy, light ones rough). Orbit the light (right-drag) and watch the highlight break up: it stays on the dark regions of the roughness map and vanishes on the light ones; the probe tells you the roughness value under any point. Then untick “Roughness” and move the “Constant roughness” slider to see the whole surface go from mirror-like to matte.

Part II

Implementation

Chapter 7

Architecture of the program

Texture Explorer is about 1 800 lines of C++20 and HLSL. It uses three small libraries: Dear ImGui for the user interface, `stb_image` to decode images, and `nlohmann/json` to read the list of materials. All three are fetched by CMake when the project is configured.

7.1 Modules

Files	Responsibility
<code>src/Mesh.h, .cpp</code>	Parametric patches of the four solids; mesh generation; tangent frames; inverting the texture mapping
<code>src/Material.h, .cpp</code>	The manifest of materials; loading images into GPU textures and CPU copies; bilinear sampling on the CPU
<code>src/Inspector.h, .cpp</code>	The probe: map values, slope, tangent-space and world-space normals at a texel
<code>shaders/pbr.hls1</code>	Displacement (vertex shader); bump mapping and GGX lighting (pixel shader); debug views
<code>shaders/gizmo.hls1</code>	Coloured lines for the arrows of the probe and the light marker
<code>src/Renderer.h, .cpp</code>	Direct3D 11 device, swap chain, shader compilation, off-screen multisampled target, draw calls
<code>src/App.h, .cpp</code>	Application state and the six panels of the user interface
<code>src/main.cpp</code>	Window, Dear ImGui set-up, main loop
<code>tools/fetch_textures.ps1</code>	Downloads the ten materials and writes <code>assets/materials/manifest.json</code>

Table 7.1: The source files of Texture Explorer.

7.2 One frame

Dear ImGui is an *immediate-mode* library: the user interface is not a tree of widget objects but a function that runs every frame and calls `ImGui::Button`, `ImGui::SliderFloat`, and so on. Each call draws a widget and tells whether the user interacted with it. The application's state therefore lives in ordinary member variables that the rest of the program reads directly. One frame proceeds as follows (Figure 7.1):

1. The animations advance by the elapsed time.
2. The panels are built. While the *Maps* and *Inspector* panels draw their images, they convert the mouse position into texture coordinates and update the *probe*.

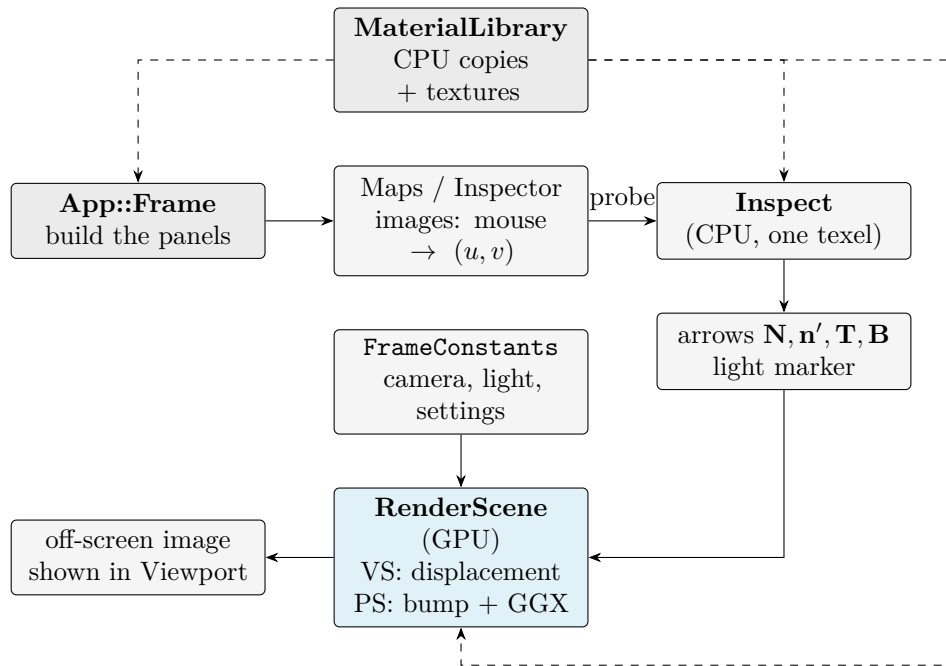


Figure 7.1: The flow of one frame. The same material data feeds the probe (on the CPU) and the renderer (on the GPU), which apply the same formulas.

3. If the probe is valid, **Inspect** computes everything about that texel, on the CPU.
4. The *Viewport* panel fills a constant buffer with the camera, the light and the settings, adds the inspector's arrows to a list of lines, and asks the renderer to draw the scene into an off-screen image of the size of the panel. It then shows that image.
5. Dear ImGui draws the whole interface into the window, and the frame is presented.

7.3 A design principle: compute it twice

The pixel shader computes the bumped normal for millions of pixels and shows only the result. To make the computation visible, the probe repeats it on the CPU for one texel and shows every intermediate value. The two implementations must agree exactly, so they are written to be identical:

- the CPU keeps a byte-for-byte copy of every map and samples it bilinearly with the GPU's texel-centre convention (Section 9.3);
- the slope is estimated with the same central differences, at level 0, with the same strength (`BumpNormalTangent` in C++, `BumpNormalTS` in HLSL);
- the tangent frame is built with the same Gram-Schmidt construction from the same patch functions (`TangentFrame` and the pixel shader);
- the 3D position of the probe comes from the same parametric functions that generated the mesh, so the arrows land on the pixels that show the probed texel.

The probe and the image disagree only where the program simplifies on purpose. For example, the probe evaluates the exact surface, while the GPU draws a mesh of flat triangles.

Chapter 8

Geometry: patches, meshes and tangent frames

This chapter implements Chapter 2. The key abstraction is the *patch*: a function from the parameter square to a surface point with everything the rest of the program needs to know about it.

8.1 Surface points, vertices and patches

A patch returns a **SurfacePoint**: position, unit normal, the derivatives $\mathbf{P}_u$ and $\mathbf{P}_v$ (not normalized), and the texture coordinates. The GPU receives a more compact **Vertex**, which stores the tangent made perpendicular to the normal, and the handedness sign σ of Chapter 5 in its fourth component:

src/Mesh.h, lines 22–27

```
22 struct Vertex {
23     DirectX::XMFLOAT3 pos;
24     DirectX::XMFLOAT3 normal;
25     DirectX::XMFLOAT4 tangent; // xyz = T orthogonalized against N, w = sign so B = w * cross
    ↪ (N, T)
26     DirectX::XMFLOAT2 uv;
27 };
```

A **Patch** wraps the function. When the patch’s texture coordinates are an affine function of its parameters, $u = u_0 + a(u_1 - u_0)$, $v = v_0 + b(v_1 - v_0)$, the patch is *invertible*: from (u, v) we can recover (a, b) and so the point. The caps of the cylinder and the base of the pyramid, textured by projection, are not invertible.

src/Mesh.h, lines 29–35

```
29 struct Patch {
30     std::function<SurfacePoint(float a, float b)> eval;
31     // Patches whose uv equals (lerp(u0,u1,a), lerp(v0,v1,b)) form a one-to-one chart
32     // of the texture and can be inverted by the inspector.
33     bool invertible = false;
34     float u0 = 0, u1 = 1, v0 = 0, v1 = 1;
35 };
```

8.2 The four solids

The sphere is a direct transcription of the formulas of Chapter 2. Note that the tangent passed to `Make` is $\mathbf{P}_u / (2\pi \sin \theta)$, which remains defined at the poles:

src/Mesh.cpp, lines 16–26

```
16 std::vector<Patch> SpherePatches() {
17     Patch side;
18     side.invertible = true;
19     side.eval = [](float a, float b) {
20         const float th = kPi * b, ph = 2 * kPi * a;
21         const float st = std::sin(th), ct = std::cos(th), sp = std::sin(ph), cp = std::cos(ph)
    ↪ ;
22         XMFLOAT3 p{st * cp, ct, -st * sp};
23         return Make(p, p, {-sp, 0, -cp}, {ct * cp, -st, -ct * sp}, a, b);
24     };
25     return {side};
26 }
```

The cylinder has three patches. The caps compute their texture coordinates from their positions (the planar projection); the sign `ny` handles the top cap (seen from above) and the bottom cap (seen from below) with one function, keeping $\mathbf{T} \times \mathbf{B} = -\mathbf{N}$ on both:

src/Mesh.cpp, lines 28–46

```
28 std::vector<Patch> CylinderPatches() {
29     constexpr float r = 0.7f, H = 1.6f;
30     Patch side;
31     side.invertible = true;
32     side.eval = [](float a, float b) {
33         const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
34         return Make({r * cp, H / 2 - b * H, -r * sp}, {cp, 0, -sp}, {-sp, 0, -cp}, {0, -1, 0},
    ↪ a, b);
35     };
36     // Caps use a planar projection, so their uv overlaps the side's chart.
37     auto cap = [](float y, float ny) {
38         Patch c;
39         c.eval = [y, ny](float a, float b) {
40             const float ph = 2 * kPi * a, x = r * b * std::cos(ph), z = -r * b * std::sin(ph);
41             return Make({x, y, z}, {0, ny, 0}, {1, 0, 0}, {0, 0, -ny}, x / (2 * r) + 0.5f, -ny
    ↪ * z / (2 * r) + 0.5f);
42         };
43         return c;
44     };
45     return {side, cap(H / 2, 1), cap(-H / 2, -1)};
46 }
```

The capsule is a surface of revolution with v proportional to arc length along the profile. For each of the three pieces of the profile the code computes the profile point (ρ, y) (radial, y), the profile normal $(\mathbf{n}_r, \mathbf{n}_y)$, and the profile's unit tangent $(d\rho/ds, dy/ds)$ ($d\mathbf{r}, d\mathbf{y}$). The revolution by the azimuth then yields the point, the normal and $\mathbf{P}_v$:

src/Mesh.cpp, lines 48–74

```
48 std::vector<Patch> CapsulePatches() {
49     constexpr float r = 0.5f, L = 0.5f;           // radius, half length of the cylinder
50     constexpr float q = kPi * r / 2;             // arc length of a quarter circle
51     constexpr float total = 2 * q + 2 * L;         // v is proportional to profile arc length
52     Patch side;
53     side.invertible = true;
54     side.eval = [](float a, float b) {
```

```

55     const float s = b * total;
56     float radial, y, nr, ny, dr, dy;           // profile point, normal, and d/ds
57     if (s < q) {
58         const float th = s / r;
59         radial = r * std::sin(th); y = L + r * std::cos(th);
60         nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
61     } else if (s < q + 2 * L) {
62         radial = r; y = L - (s - q);
63         nr = 1; ny = 0; dr = 0; dy = -1;
64     } else {
65         const float th = kPi / 2 + (s - q - 2 * L) / r;
66         radial = r * std::sin(th); y = -L + r * std::cos(th);
67         nr = std::sin(th); ny = std::cos(th); dr = std::cos(th); dy = -std::sin(th);
68     }
69     const float ph = 2 * kPi * a, sp = std::sin(ph), cp = std::cos(ph);
70     return Make({radial * cp, y, -radial * sp}, {nr * cp, ny, -nr * sp}, {-sp, 0, -cp},
71                {dr * cp, dy, -dr * sp}, a, b);
72 };
73 return {side};
74 }

```

Each side face of the pyramid is the family of segments from the apex to the points of the base edge. The face normal is computed once, as $\mathbf{B} \times \mathbf{T}$ (our orientation rule read backwards), from the edge and the median of the face:

src/Mesh.cpp, lines 76–107

```

76 std::vector<Patch> PyramidPatches() {
77     constexpr float a = 0.8f, h = 0.8f;
78     static const XMFLOAT3 apex{0, h, 0};
79     static const XMFLOAT3 c[4] = {{-a, -h, a}, {a, -h, a}, {a, -h, -a}, {-a, -h, -a}};
80     std::vector<Patch> patches;
81     // Each side face owns a quarter of the texture in u; v runs from apex (0) to base (1).
82     for (int i = 0; i < 4; ++i) {
83         Patch f;
84         f.invertible = true;
85         f.u0 = i / 4.0f; f.u1 = (i + 1) / 4.0f;
86         f.eval = [i](float s, float t) {
87             const XMVECTOR A = XMLoadFloat3(&apex), C0 = XMLoadFloat3(&c[i]), C1 =
88             ↪ XMLoadFloat3(&c[(i + 1) % 4]);
89             const XMVECTOR edge = XMVectorLerp(C0, C1, s);
90             const XMVECTOR P = A + t * (edge - A);
91             const XMVECTOR du = C1 - C0, dv = edge - A;
92             const XMVECTOR N = XMVector3Normalize(XMVector3Cross(0.5f * (C0 + C1) - A, du));
93             SurfacePoint sp;
94             XMStoreFloat3(&sp.pos, P); XMStoreFloat3(&sp.normal, N);
95             XMStoreFloat3(&sp.dPdu, du); XMStoreFloat3(&sp.dPdv, dv);
96             sp.uv = {(i + s) / 4.0f, t};
97             return sp;
98         };
99         patches.push_back(f);
100     }
101     Patch base;
102     base.eval = [](float s, float t) {
103         const float x = -a + 2 * a * s, z = a - 2 * a * t;
104         return Make({x, -h, z}, {0, -1, 0}, {1, 0, 0}, {0, 0, -1}, s, t);
105     };
106     patches.push_back(base);
107     return patches;

```

8.3 The tangent frame

`TangentFrame` implements the Gram–Schmidt construction of Chapter 5. If $\mathbf{P}_u$ is degenerate (zero, or parallel to $\mathbf{N}$) it derives a tangent from $\mathbf{P}_v$ instead. The bitangent is $\pm \mathbf{N} \times \mathbf{T}$, with the sign that makes it point towards $+v$:

src/Mesh.cpp, lines 130–140

```
130 void TangentFrame(const SurfacePoint& p, XMFLOAT3& T, XMFLOAT3& B, XMFLOAT3& N) {
131     const XMVECTOR n = XMVector3Normalize(XMLoadFloat3(&p.normal));
132     const XMVECTOR du = XMLoadFloat3(&p.dPdu), dv = XMLoadFloat3(&p.dPdv);
133     XMVECTOR t = du - n * XMVector3Dot(n, du);
134     if (XMVectorGetX(XMVector3LengthSq(t)) < 1e-12f) // degenerate: derive from dv instead
135         t = XMVector3Cross(dv, n);
136     t = XMVector3Normalize(t);
137     XMVECTOR b = XMVector3Cross(n, t);
138     if (XMVectorGetX(XMVector3Dot(b, dv)) < 0) b = -b;
139     XMStoreFloat3(&T, t); XMStoreFloat3(&B, b); XMStoreFloat3(&N, n);
140 }
```

8.4 Building the mesh

`BuildMesh` samples every patch on a regular grid of $(segments+1)^2$ vertices and emits two triangles per cell. Patches do not share vertices: along seams and creases this gives each side its own texture coordinates and normals, which is exactly what is needed (and is also why displacement opens cracks there, Section 4.1). The handedness w is computed for each vertex and stored with the tangent.

src/Mesh.cpp, lines 142–166

```
142 MeshData BuildMesh(Shape shape, int segments) {
143     MeshData mesh;
144     for (const Patch& patch : MakePatches(shape)) {
145         const uint32_t base = static_cast<uint32_t>(mesh.vertices.size());
146         for (int j = 0; j <= segments; ++j) {
147             for (int i = 0; i <= segments; ++i) {
148                 const SurfacePoint sp = patch.eval(float(i) / segments, float(j) / segments);
149                 XMFLOAT3 T, B, N;
150                 TangentFrame(sp, T, B, N);
151                 // Handedness: B = w * cross(N, T)
152                 const XMVECTOR nxT = XMVector3Cross(XMLoadFloat3(&N), XMLoadFloat3(&T));
153                 const float w = XMVectorGetX(XMVector3Dot(nxT, XMLoadFloat3(&B))) < 0 ? -1.0f
154                 ↪ : 1.0f;
155                 mesh.vertices.push_back({sp.pos, N, {T.x, T.y, T.z, w}, sp.uv});
156             }
157         }
158         const uint32_t row = segments + 1;
159         for (int j = 0; j < segments; ++j) {
160             for (int i = 0; i < segments; ++i) {
161                 const uint32_t i0 = base + j * row + i, i1 = i0 + 1, i2 = i0 + row, i3 = i2 +
162                 ↪ 1;
163                 mesh.indices.insert(mesh.indices.end(), {i0, i2, i1, i1, i2, i3});
164             }
165         }
166     }
167     return mesh;
168 }
```

8.5 Inverting the texture mapping

Finally, the function that the probe uses to find the texel on the solid: look for an invertible patch whose rectangle in texture space contains (u, v) , solve the affine equations for the parameters, and evaluate the patch.

src/Mesh.cpp, lines 168–176

```
168 bool FindSurfacePoint(Shape shape, float u, float v, SurfacePoint& out) {
169     for (const Patch& patch : MakePatches(shape)) {
170         if (!patch.invertible) continue;
171         if (u < patch.u0 || u > patch.u1 || v < patch.v0 || v > patch.v1) continue;
172         out = patch.eval((u - patch.u0) / (patch.u1 - patch.u0), (v - patch.v0) / (patch.v1 -
    ↪ patch.v0));
173         return true;
174     }
175     return false;
176 }
```

Chapter 9

Materials and textures

9.1 Collecting the materials

The ten example materials come from two libraries of physically based materials released under the CC0 licence (public domain): seven from *ambientCG* and three from *Poly Haven* (Appendix A). The script `tools/fetch_textures.ps1` downloads them:

- For ambientCG it asks the web API once for all seven assets (https://ambientcg.com/api/v2/full_json?id=...&include=downloadData), downloads the 1K-JPG ZIP archive of each, and extracts the colour, displacement, roughness and DirectX normal maps.
- For Poly Haven it asks <https://api.polyhaven.com/files/<id>> for the address of each map at 1K resolution, and <https://api.polyhaven.com/info/<id>> for the name and authors.
- It renames every map to a standard name (`color.jpg`, `displacement.jpg`, `roughness.jpg`, `normal.jpg`) and writes a *manifest*.

The manifest records each material’s origin, so the program can credit its source:

```
{
  "id": "Bricks076C", "name": "Bricks 076 C", "site": "ambientCG",
  "url": "https://ambientcg.com/view?id=Bricks076C",
  "author": "Lennart Demes (ambientCG)", "license": "CC0 1.0",
  "files": { "color": "Bricks076C/color.jpg", "displacement": "Bricks076C/displacement.jpg",
            "roughness": "Bricks076C/roughness.jpg", "normal": "Bricks076C/normal.jpg" },
  "bumpFromHeight": true
}
```

Neither library ships a separate *bump* map for these materials, and that is no accident: a bump map *is* a height field, and the displacement map is the height field of the material. The program therefore uses the displacement map as the bump map when no bump map is given, and labels it “Bump (from height)” in the interface.

9.2 Two copies of every map

Each map is kept twice. The GPU copy is a texture with a complete mipmap chain, used for rendering and for display in the interface. The CPU copy, an array of bytes, is what the probe reads. Keeping it costs 4 MB per 1024×1024 map, a small price for being able to compute on the CPU exactly what the GPU computes.

src/Material.h, lines 16–27

```

16 struct MapImage {
17     int width = 0, height = 0;
18     std::vector<uint8_t> rgba;           // CPU copy, 8 bits per channel
19     ComPtr<ID3D11Texture2D> texture;
20     ComPtr<ID3D11ShaderResourceView> srvLinear; // raw values (data maps, UI display)
21     ComPtr<ID3D11ShaderResourceView> srvSrgb;   // color decoded from sRGB (shading)
22
23     // Bilinear sample with wrap addressing; channels in [0,1].
24     DirectX::XMFLOAT4 Sample(float u, float v) const;
25     // Nearest texel fetch with wrap addressing (red channel).
26     float Texel(int x, int y) const;
27 };

```

9.3 Sampling on the CPU

`MapImage::Sample` is the bilinear filter of Chapter 3, with wrap addressing and the GPU's texel-centre convention (the -0.5):

src/Material.cpp, lines 25–40

```

25 XMFLOAT4 MapImage::Sample(float u, float v) const {
26     // Same convention as the GPU: texel centers sit at (i + 0.5) / size.
27     const float x = u * width - 0.5f, y = v * height - 0.5f;
28     const int x0 = int(std::floor(x)), y0 = int(std::floor(y));
29     const float fx = x - x0, fy = y - y0;
30     float out[4] = {};
31     for (int c = 0; c < 4; ++c) {
32         auto at = [&](int xi, int yi) {
33             return rgba[(size_t(Wrap(yi, height)) * width + Wrap(xi, width)) * 4 + c] / 255.0f
34             ↪ ;
35         };
36         const float top = at(x0, y0) * (1 - fx) + at(x0 + 1, y0) * fx;
37         const float bot = at(x0, y0 + 1) * (1 - fx) + at(x0 + 1, y0 + 1) * fx;
38         out[c] = top * (1 - fy) + bot * fy;
39     }
40     return {out[0], out[1], out[2], out[3]};
41 }

```

9.4 Loading an image into a typeless texture

`LoadImage` decodes the file with `stb_image`, always asking for four channels (a grey JPEG becomes grey RGB, which keeps the rest of the code uniform), and creates the texture. The important details are in the texture description:

- `DXGI_FORMAT_R8G8B8A8_TYPELESS` allows two views with different interpretations of the same bytes: `UNORM` returns the stored values; `UNORM_SRGB` decodes sRGB to linear (Section 3.4). Only the colour map gets an sRGB view; data maps must not be decoded.
- `MipLevels = 0` requests the full chain, and the flags `RENDER_TARGET` and `GENERATE_MIPS` let the GPU compute the smaller levels with `GenerateMips`.

src/Material.cpp, lines 42–85

```

42 static std::shared_ptr<MapImage> LoadImage(const std::filesystem::path& path, bool srgb,
43                                             ID3D11Device* device, ID3D11DeviceContext* ctx) {

```

```

44     FILE* f = nullptr;
45     _wfopen_s(&f, path.c_str(), L"rb");
46     if (!f) return nullptr;
47     int w = 0, h = 0, n = 0;
48     stbi_uc* pixels = stbi_load_from_file(f, &w, &h, &n, 4);
49     fclose(f);
50     if (!pixels) return nullptr;
51
52     auto img = std::make_shared<MapImage>();
53     img->width = w;
54     img->height = h;
55     img->rgba.assign(pixels, pixels + size_t(w) * h * 4);
56     stbi_image_free(pixels);
57
58     // Typeless storage lets the same texture be viewed as linear (UI) or sRGB (shading).
59     D3D11_TEXTURE2D_DESC td{};
60     td.Width = w;
61     td.Height = h;
62     td.MipLevels = 0; // full chain
63     td.ArraySize = 1;
64     td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
65     td.SampleDesc.Count = 1;
66     td.Usage = D3D11_USAGE_DEFAULT;
67     td.BindFlags = D3D11_BIND_SHADER_RESOURCE | D3D11_BIND_RENDER_TARGET;
68     td.MiscFlags = D3D11_RESOURCE_MISC_GENERATE_MIPS;
69     if (FAILED(device->CreateTexture2D(&td, nullptr, &img->texture))) return nullptr;
70
71     D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
72     sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
73     sd.Texture2D.MipLevels = UINT(-1);
74     sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
75     device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvLinear);
76     if (srgb) {
77         sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
78         device->CreateShaderResourceView(img->texture.Get(), &sd, &img->srvSrgb);
79     } else {
80         img->srvSrgb = img->srvLinear;
81     }
82     ctx->UpdateSubresource(img->texture.Get(), 0, nullptr, img->rgba.data(), w * 4, 0);
83     ctx->GenerateMips(img->srvLinear.Get());
84     return img;
85 }

```

9.5 Loading a material

A material's maps are loaded the first time it is needed; the program loads all ten at start-up so that the list can show thumbnails. Note how the bump map falls back to the displacement map:

src/Material.cpp, lines 120–135

```

120 void MaterialLibrary::EnsureLoaded(int index, ID3D11Device* device, ID3D11DeviceContext* ctx)
121     ↪ {
122     Material& m = items_[index];
123     if (m.loaded) return;
124     m.loaded = true;
125     for (int k = 0; k < MapCount; ++k)
126         if (!m.files[k].empty())
127             m.maps[k] = LoadImage(dir_ / std::filesystem::path(reinterpret_cast<const char8_t
128             ↪ *>(m.files[k].c_str()))), k == MapColor, device, ctx);
129     // A bump map is a height field used only for shading. When a pack has no separate
130     // one, the height (displacement) map is the bump map.

```

```
129     if (!m.maps[MapBump]) {
130         m.maps[MapBump] = m.maps[MapDisplacement];
131         m.bumpFromHeight = true;
132     } else {
133         m.bumpFromHeight = false;
134     }
135 }
```

Chapter 10

The probe

The probe is the reason Texture Explorer exists. Given the current material, a texel (u, v) , the shape and the rotation of the solid, and the bump settings, it computes everything that Chapters 4 and 5 talk about, and the Inspector panel shows it.

10.1 The normal of the bump map

BumpNormalTangent implements equation (4.3): four bilinear samples one texel around the point, central differences, and the normal $(-kh_x, -kh_y, 1)$ normalized. It also returns the slope so that the Inspector can show it.

src/Inspector.cpp, lines 5–15

```
5 XMFLOAT3 BumpNormalTangent(const MapImage& bump, float u, float v, float strength, XMFLOAT2*  
    ↪ slopeOut) {  
6     const float du = 1.0f / bump.width, dv = 1.0f / bump.height;  
7     // Height change per texel, by central differences.  
8     const float dhdx = 0.5f * (bump.Sample(u + du, v).x - bump.Sample(u - du, v).x);  
9     const float dhdy = 0.5f * (bump.Sample(u, v + dv).x - bump.Sample(u, v - dv).x);  
10    if (slopeOut) *slopeOut = {dhdx, dhdy};  
11    // The surface  $z = k*h(x,y)$  has normal  $(-k*dh/dx, -k*dh/dy, 1)$ .  
12    XMFLOAT3 n;  
13    XMStoreFloat3(&n, XMVector3Normalize(XMVectorSet(-strength * dhdx, -strength * dhdy, 1, 0)  
    ↪ ));  
14    return n;  
15 }
```

10.2 From the texel to the world

Inspect first samples every map at the probe. Then it finds the probed texel on the solid (FindSurfacePoint), builds the tangent frame there, and multiplies the tangent-space normal by the TBN matrix. The position is displaced like the vertex shader displaces vertices, so the arrows start on the surface the user sees. Finally everything is rotated into the world by the world matrix, which is a pure rotation and therefore also transforms normals correctly (Section 5.4).

src/Inspector.cpp, lines 17–47

```
17 HoverInfo Inspect(const Material& m, float u, float v, Shape shape, FXMMATRIX world, const  
    ↪ BumpSettings& s) {  
18     HoverInfo h;  
19     h.valid = true;
```

```

20     h.uv = {u, v};
21     if (auto& c = m.maps[MapColor]) { const XMFLOAT4 x = c->Sample(u, v); h.color = {x.x, x.y,
    ↪   x.z}; }
22     if (auto& b = m.maps[MapBump]) h.bump = b->Sample(u, v).x;
23     if (auto& d = m.maps[MapDisplacement]) h.displacement = d->Sample(u, v).x;
24     if (auto& r = m.maps[MapRoughness]) h.roughness = r->Sample(u, v).x;
25     h.normalTangent = m.maps[MapBump] ? BumpNormalTangent(*m.maps[MapBump], u, v, s.strength,
    ↪   &h.slope)
26                                     : XMFLOAT3{0, 0, 1};
27
28     SurfacePoint sp;
29     if (!FindSurfacePoint(shape, u, v, sp)) return h;
30     h.onSurface = true;
31     XMFLOAT3 T, B, N;
32     TangentFrame(sp, T, B, N);
33     const XMVECTOR t = XMLoadFloat3(&T), b = XMLoadFloat3(&B), n = XMLoadFloat3(&N);
34     // Tangent space -> object space: columns of the TBN matrix are T, B, N.
35     const XMFLOAT3& nt = h.normalTangent;
36     const XMVECTOR nObj = t * nt.x + b * nt.y + n * nt.z;
37     XMVECTOR p = XMLoadFloat3(&sp.pos);
38     if (s.useDisplacement && m.maps[MapDisplacement])
39         p += n * (s.displacementScale * (h.displacement - s.displacementMid));
40     // Object -> world. The world matrix is a pure rotation, so it also transforms normals.
41     XMStoreFloat3(&h.position, XMVector3TransformCoord(p, world));
42     XMStoreFloat3(&h.normalGeom, XMVector3Normalize(XMVector3TransformNormal(n, world)));
43     XMStoreFloat3(&h.normalBumped, XMVector3Normalize(XMVector3TransformNormal(nObj, world)));
44     XMStoreFloat3(&h.tangent, XMVector3Normalize(XMVector3TransformNormal(t, world)));
45     XMStoreFloat3(&h.bitangent, XMVector3Normalize(XMVector3TransformNormal(b, world)));
46     return h;
47 }

```

10.3 What the Inspector shows

The Inspector lists, for the probed texel:

u, v the normalized coordinates and the integer texel;

Color (sRGB) the stored colour, with a swatch;

Bump value, Displacement, Spec. roughness as fractions and as stored bytes (/255);

slope h_x and h_y in height units per texel;

Tangent space n_{ts} equation (4.3), with its colour encoding $\frac{1}{2}\mathbf{n} + \frac{1}{2}$;

Pack normal map the material's own normal map at the same texel, decoded by $2\text{rgb} - 1$, for comparison (Section 4.4);

World $\mathbf{n}'$, World $\mathbf{N}$, World $\mathbf{T}$, World $\mathbf{B}$, World position the bumped and the geometric normals, the frame, and the displaced point, in world coordinates.

The same quantities appear in the 3D view as arrows (Figure 1.1). The bumped normal $\mathbf{n}'$ deviates from $\mathbf{N}$ by an angle that grows with the bump strength, and the frame $\mathbf{T}, \mathbf{B}$ shows how the texture is laid on the surface at that point.

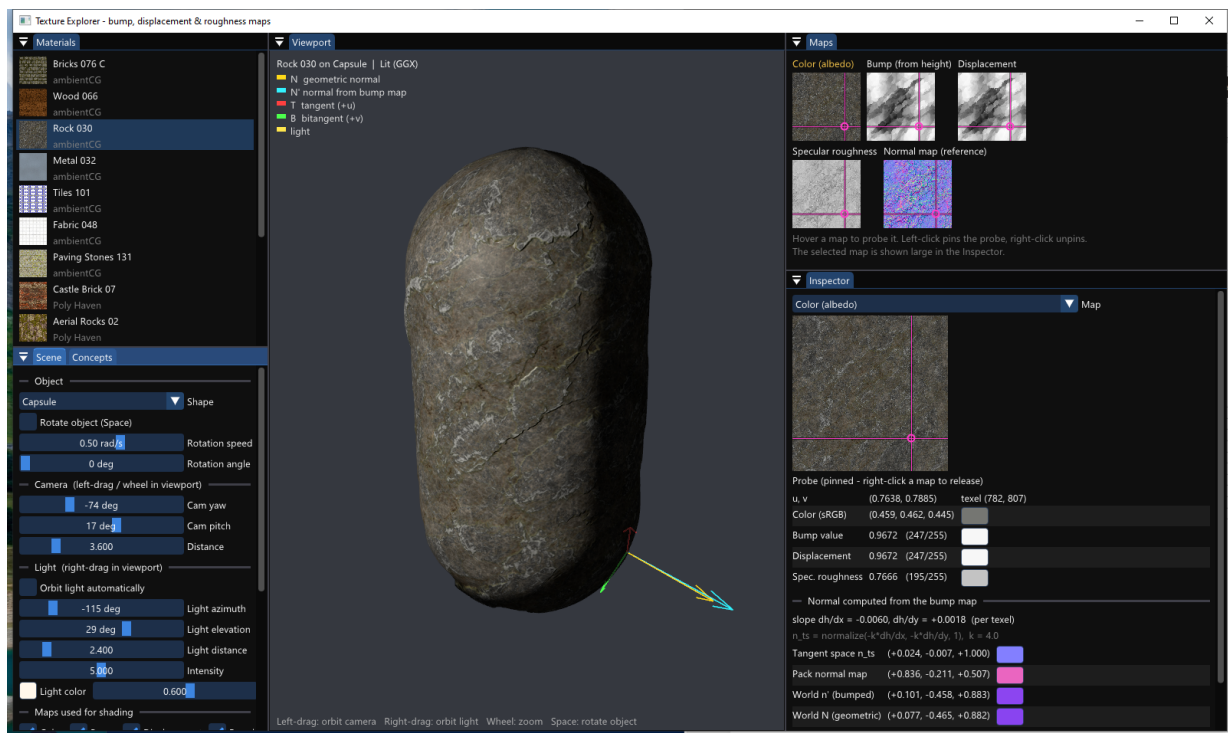


Figure 10.1: The rock material on the capsule, with a pinned probe. Compare the “Tangent space n_{ts} ” row, computed from the bump map, with the “Pack normal map” row, decoded from the material’s own normal map: the material’s map was baked from much finer geometry, so here it leans far more strongly.

Chapter 11

The shaders

The GPU work is done by two short HLSL programs in `shaders/pbr.hlsl`. The program compiles them at start-up and whenever the user presses F5, so students can edit the file while the program runs. If a modified shader does not compile, the program keeps the previous version and shows the compiler's messages in the Scene panel.

11.1 The constant buffer

All per-frame parameters travel in one constant buffer. HLSL packs constant buffers in rows of 16 bytes and does not let a vector straddle two rows, so the declaration is arranged in groups of four scalars:

`shaders/pbr.hlsl`, lines 4–15

```
4 cbuffer Frame : register(b0)
5 {
6     row_major float4x4 world;      // object -> world (a rotation)
7     row_major float4x4 viewProj;   // world -> clip
8     float3 cameraPos;   float bumpStrength;
9     float3 lightPos;    float lightIntensity;
10    float3 lightColor;   float dispScale;
11    float dispMid;       float dispLod;   float metallic;   float ambient;
12    int  viewMode;       int useColor;    int useBump;      int useDisp;
13    int  useRough;       float constRoughness; float2 hoverUV;
14    float hoverOn;       float3 pad;
15 };
```

The C++ structure that fills it must have exactly the same layout; a `static_assert` checks the most common mistake, a size that is not a multiple of 16:

`src/Renderer.h`, lines 12–22

```
12 struct FrameConstants {
13     DirectX::XMFLOAT4X4 world;
14     DirectX::XMFLOAT4X4 viewProj;
15     DirectX::XMFLOAT3 cameraPos;   float bumpStrength;
16     DirectX::XMFLOAT3 lightPos;    float lightIntensity;
17     DirectX::XMFLOAT3 lightColor;   float dispScale;
18     float dispMid, dispLod, metallic, ambient;
19     int viewMode, useColor, useBump, useDisp;
20     int useRough; float constRoughness; DirectX::XMFLOAT2 hoverUV;
21     float hoverOn; DirectX::XMFLOAT3 pad;
22 };
```

DirectXMath stores matrices by rows and multiplies row vectors on the left, $\mathbf{p}' = \mathbf{p}M$. Declaring the HLSL matrices `row_major` and writing `mul(p, M)` follows the same convention, so no transposition is needed.

11.2 Vertex shader: displacement

The vertex shader is equation (4.1). It samples the displacement map at the mipmap level `dispLod` chosen on the CPU (Chapter 3), moves the vertex along its normal, and transforms position, normal and tangent to the world. It does *not* recompute the normal (Section 4.1).

shaders/pbr.hlsl, lines 26–43

```

26 VSOut VSMain(VSIn i)
27 {
28     float3 p = i.pos;
29     if (useDisp != 0)
30     {
31         // Displacement mapping: move the vertex along its normal by the height.
32         float h = dispMap.SampleLevel(linearWrap, i.uv, dispLod).r;
33         p += i.nrm * dispScale * (h - dispMid);
34     }
35     VSOut o;
36     float4 wp = mul(float4(p, 1), world);
37     o.svpos = mul(wp, viewProj);
38     o.wpos = wp.xyz;
39     o.nrm = mul(float4(i.nrm, 0), world).xyz;
40     o.tan = float4(mul(float4(i.tan.xyz, 0), world).xyz, i.tan.w);
41     o.uv = i.uv;
42     return o;
43 }
```

11.3 The bump normal on the GPU

This is equation (4.3) again, written to match the C++ version line by line:

shaders/pbr.hlsl, lines 45–57

```

45 // Normal of the height field  $z = k \cdot h(x, y)$ , in tangent space, by central differences
46 // (one texel each side). Same formula as BumpNormalTangent() in src/Inspector.cpp.
47 float3 BumpNormalTS(float2 uv)
48 {
49     float w, h;
50     bumpMap.GetDimensions(w, h);
51     float2 t = 1.0 / float2(w, h);
52     float dhdx = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(t.x, 0), 0).r -
53                        bumpMap.SampleLevel(linearWrap, uv - float2(t.x, 0), 0).r);
54     float dhdy = 0.5 * (bumpMap.SampleLevel(linearWrap, uv + float2(0, t.y), 0).r -
55                        bumpMap.SampleLevel(linearWrap, uv - float2(0, t.y), 0).r);
56     return normalize(float3(-bumpStrength * dhdx, -bumpStrength * dhdy, 1));
57 }
```

11.4 Pixel shader: bump mapping and lighting

The pixel shader proceeds in the order of the theory chapters:

1. Rebuild an orthonormal frame from the interpolated normal and tangent (Gram–Schmidt), and the bitangent from the stored sign (Chapter 5).
2. Compute $\mathbf{n}_{ts}$ from the bump map and transform it with the TBN matrix.
3. Read the albedo (decoded from sRGB by the view) and the roughness, or the neutral values chosen in the interface when a map is switched off.
4. Either output a debug view or evaluate the GGX/Cook–Torrance model of Chapter 6, with the horizon factor.
5. Draw a magenta ring around the probed texel, in texture space.

shaders/pbr.hlsl, lines 63–126

```

63 float4 PSMain(VSOut i) : SV_Target
64 {
65     // Tangent frame (TBN). B is rebuilt from N and T with the stored handedness.
66     float3 N = normalize(i.nrm);
67     float3 T = normalize(i.tan.xyz - N * dot(N, i.tan.xyz));
68     float3 B = i.tan.w * cross(N, T);
69
70     float3 nTS = useBump != 0 ? BumpNormalTS(i.uv) : float3(0, 0, 1);
71     float3 n = normalize(T * nTS.x + B * nTS.y + N * nTS.z);
72
73     float3 albedo = useColor != 0 ? colorMap.Sample(linearWrap, i.uv).rgb : float3(0.7, 0.7,
74     ↪ 0.7);
75     float rough = useRough != 0 ? roughMap.Sample(linearWrap, i.uv).r : constRoughness;
76     rough = clamp(rough, 0.04, 1.0);
77
78     float3 dbg = float3(-1, -1, -1);
79     if (viewMode == 1) dbg = pow(saturate(albedo), 1 / 2.2);
80     else if (viewMode == 2) dbg = n * 0.5 + 0.5;
81     else if (viewMode == 3) dbg = nTS * 0.5 + 0.5;
82     else if (viewMode == 4) dbg = rough.xxx;
83     else if (viewMode == 5) dbg = bumpMap.Sample(linearWrap, i.uv).rrr;
84     else if (viewMode == 6) dbg = float3(frac(i.uv), 0);
85
86     float3 color;
87     if (dbg.x >= 0)
88     {
89         color = SrgbToLinear(dbg);
90     }
91     else
92     {
93         // Cook-Torrance microfacet BRDF: GGX distribution, Smith-Schlick geometry, Schlick
94         ↪ Fresnel.
95         float3 V = normalize(cameraPos - i.wpos);
96         float3 L = normalize(lightPos - i.wpos);
97         float3 H = normalize(V + L);
98         float NdotL = saturate(dot(n, L));
99         float NdotV = max(dot(n, V), 1e-4);
100        float NdotH = saturate(dot(n, H));
101        float VdotH = saturate(dot(V, H));
102
103        float a = rough * rough;
104        float a2 = a * a;
105        float d = NdotH * NdotH * (a2 - 1) + 1;
106        float D = a2 / (PI * d * d);
107        float k = (rough + 1) * (rough + 1) / 8;
108        float G = (NdotV / (NdotV * (1 - k) + k)) * (NdotL / (NdotL * (1 - k) + k));
109        float3 F0 = lerp(float3(0.04, 0.04, 0.04), albedo, metallic);
110        float3 F = F0 + (1 - F0) * pow(1 - VdotH, 5);

```

```

110     float3 spec = D * G * F / (4 * NdotV * max(NdotL, 1e-4));
111     float3 kd = (1 - F) * (1 - metallic);
112     float3 radiance = lightColor * lightIntensity;
113     // Geometric self-shadowing: no light where the unperturbed surface faces away.
114     float horizon = saturate(dot(N, L) * 8);
115     color = (kd * albedo / PI + spec) * radiance * NdotL * horizon + ambient * albedo;
116 }
117
118 // Highlight the texel under the mouse in the inspector.
119 if (hoverOn > 0)
120 {
121     float2 duv = abs(i.uv - hoverUV);
122     float r = length(duv);
123     if (r > 0.010 && r < 0.016) color = float3(1, 0.1, 0.8);
124 }
125 return float4(color, 1);
126 }

```

Two details deserve a comment. The debug views write values such as $\frac{1}{2}\mathbf{n} + \frac{1}{2}$ that must reach the screen unchanged, but the render target encodes everything as sRGB; the shader therefore applies the approximate inverse, $c^{2.2}$, beforehand. And the roughness is clamped to at least 0.04, because a perfectly smooth GGX surface has an infinitely thin highlight that a point light cannot show.

Chapter 12

The renderer and the user interface

12.1 Shaders that can be reloaded

ReloadShaders compiles the four entry points from their files with D3DCompileFromFile, collecting the compiler's messages in a log. It creates the new shader objects and input layouts first, and replaces the old ones only if everything succeeded. A typing mistake in a shader therefore leaves the program running with the previous version. The input layouts use `offsetof` on the C++ vertex structures, so the structures are the only description of the vertex format.

src/Renderer.cpp, lines 148–181

```
148 bool Renderer::ReloadShaders(const std::filesystem::path& dir, std::string& log) {
149     log.clear();
150     auto vs = Compile(dir / "pbr.hlsl", "VSMain", "vs_5_0", log);
151     auto ps = Compile(dir / "pbr.hlsl", "PSMain", "ps_5_0", log);
152     auto lvs = Compile(dir / "gizmo.hlsl", "VSLine", "vs_5_0", log);
153     auto lps = Compile(dir / "gizmo.hlsl", "PSLine", "ps_5_0", log);
154     if (!vs || !ps || !lvs || !lps) return false; // keep the previous working shaders
155
156     const D3D11_INPUT_ELEMENT_DESC meshElems[] = {
157         {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, pos),
158         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
159         {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(Vertex, normal),
160         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
161         {"TANGENT", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(Vertex, tangent),
162         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
163         {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, offsetof(Vertex, uv),
164         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
165     };
166     const D3D11_INPUT_ELEMENT_DESC lineElems[] = {
167         {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, offsetof(LineVertex, pos),
168         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
169         {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, offsetof(LineVertex, color),
170         ↪ D3D11_INPUT_PER_VERTEX_DATA, 0},
171     };
172     ComPtr<ID3D11VertexShader> newVs, newLineVs;
173     ComPtr<ID3D11PixelShader> newPs, newLinePs;
174     ComPtr<ID3D11InputLayout> newLayout, newLineLayout;
175     if (FAILED(device->CreateVertexShader(vs->GetBufferPointer(), vs->GetBufferSize(),
176     ↪ nullptr, &newVs)) ||
177         FAILED(device->CreatePixelShader(ps->GetBufferPointer(), ps->GetBufferSize(), nullptr
178     ↪ , &newPs)) ||
179         FAILED(device->CreateVertexShader(lvs->GetBufferPointer(), lvs->GetBufferSize(),
180     ↪ nullptr, &newLineVs)) ||
181         FAILED(device->CreatePixelShader(lps->GetBufferPointer(), lps->GetBufferSize(),
182     ↪ nullptr, &newLinePs)) ||
```

```

173     FAILED(device_>CreateInputLayout(meshElems, 4, vs->GetBufferPointer(), vs->
↪ GetBufferSize(), &newLayout)) ||
174     FAILED(device_>CreateInputLayout(lineElems, 2, lvs->GetBufferPointer(), lvs->
↪ GetBufferSize(), &newLineLayout))) {
175     log += "Failed to create shader objects\n";
176     return false;
177 }
178 meshVs_ = newVs; meshPs_ = newPs; lineVs_ = newLineVs; linePs_ = newLinePs;
179 meshLayout_ = newLayout; lineLayout_ = newLineLayout;
180 return true;
181 }

```

12.2 The off-screen target

The 3D view is rendered into a texture the size of the Viewport panel, which Dear ImGui then shows like any other image. The target uses four samples per pixel (multisample antialiasing), which smooths the silhouette of the solid and the thin lines of the probe. It is written through an sRGB view, so the GPU encodes the linear colours of the lighting, and *resolved* into a single-sample texture that the interface reads through a plain view. This is the same typeless-texture technique used for the colour maps (Section 9.4).

src/Renderer.cpp, lines 183–216

```

183 void Renderer::EnsureSceneTarget(UINT width, UINT height) {
184     if (width == sceneW_ && height == sceneH_ && msaaColor_) return;
185     sceneW_ = width;
186     sceneH_ = height;
187     // Typeless so it can be rendered as sRGB (gamma-correct lighting) but shown by
188     // ImGui as UNORM, i.e. the already-encoded values reach the screen untouched.
189     D3D11_TEXTURE2D_DESC td{};
190     td.Width = width;
191     td.Height = height;
192     td.MipLevels = td.ArraySize = 1;
193     td.Format = DXGI_FORMAT_R8G8B8A8_TYPELESS;
194     td.SampleDesc.Count = 4;
195     td.BindFlags = D3D11_BIND_RENDER_TARGET;
196     device_>CreateTexture2D(&td, nullptr, &msaaColor_);
197     D3D11_RENDER_TARGET_VIEW_DESC rd{};
198     rd.Format = DXGI_FORMAT_R8G8B8A8_UNORM_SRGB;
199     rd.ViewDimension = D3D11_RTV_DIMENSION_TEXTURE2DMS;
200     device_>CreateRenderTargetView(msaaColor_.Get(), &rd, &msaaRtv_);
201
202     td.SampleDesc.Count = 1;
203     td.BindFlags = D3D11_BIND_SHADER_RESOURCE;
204     device_>CreateTexture2D(&td, nullptr, &resolved_);
205     D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
206     sd.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
207     sd.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
208     sd.Texture2D.MipLevels = 1;
209     device_>CreateShaderResourceView(resolved_.Get(), &sd, &resolvedSrv_);
210
211     td.Format = DXGI_FORMAT_D32_FLOAT;
212     td.SampleDesc.Count = 4;
213     td.BindFlags = D3D11_BIND_DEPTH_STENCIL;
214     device_>CreateTexture2D(&td, nullptr, &depth_);
215     device_>CreateDepthStencilView(depth_.Get(), nullptr, &dsv_);
216 }

```

12.3 Drawing the scene

`RenderScene` clears the target, uploads the constant buffer (with `WRITE_DISCARD`, so the CPU never waits for the GPU to finish the previous frame), and draws the surface with one indexed draw call. The four maps are bound to both shader stages: the vertex shader needs the displacement map. A missing map is replaced by a neutral gray texture.

The lines of the probe and of the light marker are drawn twice from the same buffer: first at 30 % opacity without the depth test, so that parts hidden behind the solid remain faintly visible, then normally with the depth test. Finally the textures are unbound, because Direct3D refuses to bind a resource for reading and writing at once, and the multisampled image is resolved. The resolve uses the sRGB format, so the samples are averaged in linear light.

src/Renderer.cpp, lines 236–310

```

236 ID3D11ShaderResourceView* Renderer::RenderScene(UINT width, UINT height, const FrameConstants&
    ↪ fc,
237                                     const Material& material, Shape shape,
238                                     const std::vector<LineVertex>& lines) {
239     if (width == 0 || height == 0 || !meshVs_) return nullptr;
240     EnsureSceneTarget(width, height);
241
242     const float clear[4] = {0.035f, 0.04f, 0.05f, 1}; // linear (sRGB target)
243     ctx_>ClearRenderTargetView(msaaRtv_.Get(), clear);
244     ctx_>ClearDepthStencilView(dsv_.Get(), D3D11_CLEAR_DEPTH, 1, 0);
245     ctx_>OMSetRenderTargets(1, msaaRtv_.GetAddressOf(), dsv_.Get());
246     const D3D11_VIEWPORT vp{0, 0, float(width), float(height), 0, 1};
247     ctx_>RSSetViewports(1, &vp);
248     ctx_>RSSetState(rasterNoCull_.Get());
249     ctx_>OMSetDepthStencilState(depthOn_.Get(), 0);
250     ctx_>OMSetBlendState(nullptr, nullptr, 0xffffffff);
251
252     D3D11_MAPPED_SUBRESOURCE ms;
253     ctx_>Map(frameCb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
254     memcpy(ms.pData, &fc, sizeof(fc));
255     ctx_>Unmap(frameCb_.Get(), 0);
256
257     // Surface
258     GpuMesh& mesh = MeshFor(shape);
259     const UINT stride = sizeof(Vertex), offset = 0;
260     ctx_>IASetInputLayout(meshLayout_.Get());
261     ctx_>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
262     ctx_>IASetVertexBuffers(0, 1, mesh.vb.GetAddressOf(), &stride, &offset);
263     ctx_>IASetIndexBuffer(mesh.ib.Get(), DXGI_FORMAT_R32_UINT, 0);
264     ctx_>VSSetShader(meshVs_.Get(), nullptr, 0);
265     ctx_>PSSetShader(meshPs_.Get(), nullptr, 0);
266     ctx_>VSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
267     ctx_>PSSetConstantBuffers(0, 1, frameCb_.GetAddressOf());
268     auto srv = [&](MapKind k, bool srgb) -> ID3D11ShaderResourceView* {
269         const auto& map = material.maps[k];
270         if (!map) return fallbackGray_.Get();
271         return srgb ? map->srvSrgb.Get() : map->srvLinear.Get();
272     };
273     ID3D11ShaderResourceView* srvs[4] = {srv(MapColor, true), srv(MapBump, false), srv(
    ↪ MapDisplacement, false),
274                                     srv(MapRoughness, false)};
275     ctx_>VSSetShaderResources(0, 4, srvs);
276     ctx_>PSSetShaderResources(0, 4, srvs);
277     ctx_>VSSetSamplers(0, 1, sampler_.GetAddressOf());
278     ctx_>PSSetSamplers(0, 1, sampler_.GetAddressOf());
279     ctx_>DrawIndexed(mesh.indexCount, 0, 0);
280

```

```

281 // Lines: a faint "x-ray" pass ignoring depth, then a depth-tested pass on top.
282 if (!lines.empty()) {
283     const UINT n = UINT(std::min<size_t>(lines.size(), kMaxLineVertices / 2));
284     ctx_>Map(lineVb_.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &ms);
285     auto* dst = static_cast<LineVertex*>(ms.pData);
286     for (UINT i = 0; i < n; ++i) {
287         dst[i] = lines[i];
288         dst[i].color.w *= 0.3f;
289         dst[n + i] = lines[i];
290     }
291     ctx_>Unmap(lineVb_.Get(), 0);
292     const UINT lstride = sizeof(LineVertex);
293     ctx_>IASetInputLayout(lineLayout_.Get());
294     ctx_>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_LINELIST);
295     ctx_>IASetVertexBuffers(0, 1, lineVb_.GetAddressOf(), &lstride, &offset);
296     ctx_>VSSetShader(lineVs_.Get(), nullptr, 0);
297     ctx_>PSSetShader(linePs_.Get(), nullptr, 0);
298     ctx_>OMSetBlendState(alphaBlend_.Get(), nullptr, 0xffffffff);
299     ctx_>OMSetDepthStencilState(depthOff_.Get(), 0);
300     ctx_>Draw(n, 0);
301     ctx_>OMSetDepthStencilState(depthOn_.Get(), 0);
302     ctx_>Draw(n, n);
303 }
304
305 ID3D11ShaderResourceView* nullSrvs[4] = {};
306 ctx_>VSSetShaderResources(0, 4, nullSrvs);
307 ctx_>PSSetShaderResources(0, 4, nullSrvs);
308 ctx_>ResolveSubresource(resolved_.Get(), 0, msaaColor_.Get(), 0,
309     ↪ DXGI_FORMAT_R8G8B8A8_UNORM_SRGB);
310 return resolvedSrv_.Get();
311 }

```

12.4 The frame of the application

`App::Frame` is the function of Figure 7.1. The order matters in one respect: the probe is decided while the map panels are built, before the viewport, which needs the probe's result to draw the arrows.

src/App.cpp, lines 114–151

```

114 void App::Frame(float dt) {
115     if (rotate_) rotAngle_ = std::fmod(rotAngle_ + rotSpeed_ * dt, 2 * kPi);
116     if (light_.autoOrbit) light_.azimuth = std::fmod(light_.azimuth + light_.speed * dt, 2 *
117     ↪ kPi);
118     if (ImGui::IsKeyPressed(ImGuiKey_F5)) ReloadShaders();
119     if (ImGui::IsKeyPressed(ImGuiKey_Space) && !ImGui::GetIO().WantTextInput) rotate_ = !
120     ↪ rotate_;
121
122     const ImGuiID dockId = ImGui::DockSpaceOverViewport(0, nullptr,
123     ↪ ImGuiDockNodeFlags_PassthruCentralNode);
124     static bool layoutBuilt = false;
125     if (!layoutBuilt) {
126         layoutBuilt = true;
127         ImGuiDockNode* node = ImGui::DockBuilderGetNode(dockId);
128         if (!node || node->IsLeafNode()) BuildDockLayout(dockId);
129     } else if (ImGui::GetFrameCount() == 2) {
130         ImGui::SetWindowFocus("Scene"); // Scene and Concepts share a dock; start on Scene
131     }
132
133     if (library_.Items().empty()) {
134         ImGui::Begin("Viewport");
135     }

```

```

132     ImGui::TextColored(ImVec4(1, 0.4f, 0.4f, 1), "%s", loadError_.c_str());
133     ImGui::End();
134     return;
135 }
136
137 // The probe is driven by whatever map image is hovered during this frame.
138 hoveredThisFrame_ = false;
139 UiMaterials();
140 UiScene();
141 UiMaps();
142 UiInspector();
143 if (!hoveredThisFrame_ && !pinned_) probeValid_ = false;
144 if (probeValid_) {
145     info_ = Inspect(Current(), probeUV_.x, probeUV_.y, shape_, World(), bump_);
146 } else {
147     info_ = HoverInfo{};
148 }
149 UiViewport();
150 UiConcepts();
151 }

```

12.5 Images that answer the mouse

Every image of a map in the interface (the tiles of the Maps panel and the large image of the Inspector) is drawn by `App::MapImage`, which also converts the mouse position into texture coordinates:

$$(u, v) = \left(\frac{p_x - m_x}{W}, \frac{p_y - m_y}{H} \right) \text{ clamped to } [0, 1]^2,$$

where $\mathbf{m}$ is the image's top-left corner on the screen and (W, H) its size. Hovering moves the probe, a left click pins it, a right click releases it. The probe's crosshair is drawn on every map image, so the eye can compare the maps at the same texel.

src/App.cpp, lines 266–300

```

266 void App::MapImage(MapKind kind, const ImVec2& size) {
267     const auto& map = Current().maps[kind];
268     if (!map) {
269         ImGui::Button("not available", size);
270         return;
271     }
272     ImGui::Image(Tex(map->srvLinear.Get()), size);
273     const ImVec2 mn = ImGui::GetItemRectMin();
274     ImDrawList* dl = ImGui::GetWindowDrawList();
275     if (ImGui::IsItemHovered()) {
276         const ImVec2 mp = ImGui::GetMousePos();
277         const XMFLT2 uv{std::clamp((mp.x - mn.x) / size.x, 0.0f, 1.0f), std::clamp((mp.y -
↪ mn.y) / size.y, 0.0f, 1.0f)};
278         hoveredThisFrame_ = true;
279         if (!pinned_) {
280             probeUV_ = uv;
281             probeValid_ = true;
282         }
283         if (ImGui::IsMouseClicked(ImGuiMouseButton_Left)) {
284             probeUV_ = uv;
285             probeValid_ = pinned_ = true;
286         }
287         if (ImGui::IsMouseClicked(ImGuiMouseButton_Right)) pinned_ = false;
288         if (ImGui::IsMouseDoubleClicked(ImGuiMouseButton_Left)) inspected_ = kind;
289     }
290     // Crosshair at the probe position, mirrored on every map image.

```

```

291     if (probeValid_) {
292         const ImVec2 p(mn.x + probeUV_.x * size.x, mn.y + probeUV_.y * size.y);
293         const ImU32 col = pinned_ ? IM_COL32(255, 60, 200, 255) : IM_COL32(255, 255, 255, 220)
        ↪ ;
294         dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), IM_COL32(0, 0, 0, 120), 3);
295         dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), IM_COL32(0, 0, 0, 120), 3);
296         dl->AddLine(ImVec2(p.x, mn.y), ImVec2(p.x, mn.y + size.y), col);
297         dl->AddLine(ImVec2(mn.x, p.y), ImVec2(mn.x + size.x, p.y), col);
298         dl->AddCircle(p, 5, col, 0, 2);
299     }
300 }

```

12.6 Filling the constant buffer

The Viewport panel computes the camera matrices (right-handed, looking at the origin) and copies the application's state into the constant buffer. A map is used only if it is switched on *and* present, and the mipmap level for the displacement is chosen to match the mesh resolution:

src/App.cpp, lines 415–437

```

415 FrameConstants fc{};
416 XMStoreFloat4x4(&fc.world, World());
417 XMStoreFloat4x4(&fc.viewProj, view * proj);
418 fc.cameraPos = eye;
419 fc.lightPos = lp;
420 fc.lightIntensity = light_.intensity;
421 fc.lightColor = {light_.color[0], light_.color[1], light_.color[2]};
422 fc.bumpStrength = bump_.strength;
423 fc.dispScale = bump_.displacementScale;
424 fc.dispMid = bump_.displacementMid;
425 // Sample the height at a mip whose resolution matches the mesh grid (avoids aliasing).
426 const auto& disp = m.maps[MapDisplacement];
427 fc.dispLod = disp ? std::max(0.0f, std::log2(float(disp->width) / Renderer::kMeshSegments)) :
        ↪ 0;
428 fc.metallic = metallic_;
429 fc.ambient = ambient_;
430 fc.viewMode = viewMode_;
431 fc.useColor = useColor_ && m.maps[MapColor];
432 fc.useBump = useBump_ && m.maps[MapBump];
433 fc.useDisp = bump_.useDisplacement && disp;
434 fc.useRough = useRough_ && m.maps[MapRoughness];
435 fc.constRoughness = constRough_;
436 fc.hoverUV = info_.uv;
437 fc.hoverOn = info_.valid ? 1.0f : 0.0f;

```

12.7 The main loop

`main.cpp` creates the window, initializes Dear ImGui (with docking, and with its layout file next to the executable), and runs the classic loop: process the window messages, measure the elapsed time, build the interface (which renders the scene off-screen), draw the interface into the window, and present it synchronized with the display.

src/main.cpp, lines 81–112

```

81 while (running) {
82     MSG msg;
83     while (PeekMessageW(&msg, nullptr, 0, 0, PM_REMOVE)) {

```

```
84     TranslateMessage(&msg);
85     DispatchMessageW(&msg);
86     if (msg.message == WM_QUIT) running = false;
87 }
88 if (!running) break;
89 if (IsIconic(hwnd)) {
90     Sleep(10);
91     continue;
92 }
93 if (g_resizeW && g_resizeH) {
94     app.GetRenderer().Resize(g_resizeW, g_resizeH);
95     g_resizeW = g_resizeH = 0;
96 }
97
98 const auto now = std::chrono::steady_clock::now();
99 const float dt = std::chrono::duration<float>(now - last).count();
100 last = now;
101
102 ImGui_ImplDX11_NewFrame();
103 ImGui_ImplWin32_NewFrame();
104 ImGui::NewFrame();
105 app.Frame(dt);
106 ImGui::Render();
107
108 const float clear[4] = {0.08f, 0.08f, 0.09f, 1};
109 app.GetRenderer().BeginFrame(clear);
110 ImGui_ImplDX11_RenderDrawData(ImGui::GetDrawData());
111 app.GetRenderer().Present(true);
112 }
```

Chapter 13

Experiments and exercises

13.1 Experiments with the program

These experiments need no programming. Each one connects something visible in the program to a section of Part I.

- E1.** *Texture space* (Chapter 2). Select the “UV coordinates” view. On each solid, locate $u = 0$, $u = 1$ and the seam; on the sphere, observe the pinching at the poles; on the cylinder, explain why the caps show the same colours as parts of the side.
- E2.** *Normalized coordinates*. Hover over the corners of the large image in the Inspector and check that (u, v) goes from $(0, 0)$ at the top left to $(1, 1)$ at the bottom right, and that the texel index is $\lfloor uW \rfloor$.
- E3.** *Bump versus displacement* (Chapter 4). With the sphere and a rough material, untick “Bump” and look at the silhouette and the lighting. Then tick “Bump” and untick “Displacement”. Then increase “Displacement scale” to 0.3 with bump off: why does the surface look strangely smooth despite its jagged outline?
- E4.** *Cracks*. With displacement on, look at the edges of the pyramid and the rims of the cylinder from close up. Explain the cracks with equation (4.1).
- E5.** *Bump strength* (Section 4.2.3). Pin the probe on a groove and read $\mathbf{n}_{ts}$ while varying the bump strength from 0 to 30. Plot the angle between $\mathbf{n}_{ts}$ and $(0, 0, 1)$ as a function of k and compare with $\arctan(k\sqrt{h_x^2 + h_y^2})$.
- E6.** *Tangent space versus world space* (Chapter 5). Pin the probe and start the rotation. Which numbers change and which do not? Then change the shape: which numbers change now?
- E7.** *Normal maps* (Section 4.4). For several texels, compare the signs of the x and y components of the computed $\mathbf{n}_{ts}$ and of the pack normal map. Do they agree? What would happen with an OpenGL normal map?
- E8.** *Roughness* (Chapter 6). Untick “Roughness” and sweep the constant roughness from 0.04 to 1 while the light orbits automatically. Relate the size of the highlight to Figure 6.2.
- E9.** *Colour spaces* (Section 3.4). Compare the Inspector’s “Color (sRGB)” value of a mid-grey texel with the value that the lighting actually uses after decoding. How large is the difference?

13.2 Programming exercises

In increasing order of difficulty. After editing a shader, press F5; after editing C++, rebuild.

- P1.** In `SpherePatches`, remove the minus signs of the z components. Describe what happens to the texture and to the Inspector's **T** and **B**, and relate it to equation (2.1) and to the sign w .
- P2.** Replace the central differences by forward differences, both in `BumpNormalTS` and in `BumpNormalTangent`. Compare the probe's normal before and after on a sharp edge.
- P3.** Add a view mode that shows the angle between the bumped normal and the geometric normal as a heat map.
- P4.** Add a view mode that shows the difference between the normal computed from the bump map and the material's normal map. (You will need to bind the normal map to a fifth texture slot.)
- P5.** Make the bump strength physically correct by dividing by the local stretch of the parametrization, $\|\mathbf{P}_u\|$ and $\|\mathbf{P}_v\|$ (Section 4.2.3). On which solid is the change most visible?
- P6.** Recompute the normal in the vertex shader from the displacement map, so that displacement alone produces correct shading.
- P7.** Close the cracks of the pyramid by displacing the vertices of the edges along the average of the normals of the two faces that meet there.
- P8.** Replace vertex displacement by hardware tessellation (hull and domain shaders) with a tessellation factor that depends on the distance to the camera.

Appendix A

The materials

All ten materials are released under the Creative Commons CC0 1.0 licence (public domain dedication), which allows any use without attribution; the program and this guide credit them anyway. Each material is used at 1K resolution (1024×1024).

Material	Source	Page
Bricks 076 C	ambientCG	https://ambientcg.com/view?id=Bricks076C
Wood 066	ambientCG	https://ambientcg.com/view?id=Wood066
Rock 030	ambientCG	https://ambientcg.com/view?id=Rock030
Metal 032	ambientCG	https://ambientcg.com/view?id=Metal032
Tiles 101	ambientCG	https://ambientcg.com/view?id=Tiles101
Fabric 048	ambientCG	https://ambientcg.com/view?id=Fabric048
Paving Stones 131	ambientCG	https://ambientcg.com/view?id=PavingStones131
Castle Brick 07	Poly Haven (Rob Tuytel)	https://polyhaven.com/a/castle_brick_07
Aerial Rocks 02	Poly Haven (Rob Tuytel)	https://polyhaven.com/a/aerial_rocks_02
Bark Willow 02	Poly Haven (Charlotte Baglioni)	https://polyhaven.com/a/bark_willow_02

Table A.1: The materials of Texture Explorer and their origin.

Appendix B

Building this document

The sources of this guide are in the `Docs` folder of the repository:

`guide.tex` the main file, with the preamble and the list of chapters;

`chapters/` one file per chapter;

`figures/` screenshots of the program (the diagrams are drawn with TikZ and pgfplots in the chapters themselves);

`excerpts.txt` the list of code excerpts: for each, a source file and a regular expression for its first line;

`build.ps1` the build script.

The build script first extracts each excerpt from the current sources into `build/excerpts`, following the braces of the function or structure that starts at the matching line. It then runs `latexmk`, `pdflatex` or `tectonic`, whichever it finds, and writes `TextureExplorer-Guide.pdf`:

```
powershell -ExecutionPolicy Bypass -File Docs\build.ps1
powershell -ExecutionPolicy Bypass -File Docs\build.ps1 -TeX C:\tools\tectonic.exe
```

Because the listings are extracted at build time, the guide always shows the code as it is. If a function is renamed, the build stops with an error that names the excerpt to fix.

The repository also contains, in `LiterateP`, the complete program written as a literate program in the style of Donald Knuth, from which every source file can be regenerated. That document follows the code; this one follows the theory.

Bibliography

- [1] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki and S. Hillaire. *Real-Time Rendering*, 4th edition. CRC Press, 2018.
- [2] J. F. Blinn. Simulation of wrinkled surfaces. *Computer Graphics (Proceedings of SIGGRAPH 78)*, 12(3):286–292, 1978.
- [3] R. L. Cook. Shade trees. *Computer Graphics (Proceedings of SIGGRAPH 84)*, 18(3):223–231, 1984.
- [4] R. L. Cook and K. E. Torrance. A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1):7–24, 1982.
- [5] E. Heitz. Understanding the masking-shadowing function in microfacet-based BRDFs. *Journal of Computer Graphics Techniques*, 3(2):48–107, 2014.
- [6] B. Karis. Real shading in Unreal Engine 4. *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*, 2013.
- [7] M. S. Mikkelsen. *Simulation of wrinkled surfaces revisited*. Master’s thesis, University of Copenhagen, 2008.
- [8] M. Pharr, W. Jakob and G. Humphreys. *Physically Based Rendering: From Theory to Implementation*, 3rd edition. Morgan Kaufmann, 2016.
- [9] C. Schlick. An inexpensive BRDF model for physically-based rendering. *Computer Graphics Forum*, 13(3):233–246, 1994.
- [10] B. Walter, S. R. Marschner, H. Li and K. E. Torrance. Microfacet models for refraction through rough surfaces. In *Proceedings of the Eurographics Symposium on Rendering*, 2007.
- [11] L. Williams. Pyramidal parametrics. *Computer Graphics (Proceedings of SIGGRAPH 83)*, 17(3):1–11, 1983.